



עזריאלי מכללה אקדמית להנדסה ירושלים
המחלקה להנדסת חשמל ואלקטרוניקה

דימוי מבעד לרקמה מפזרת

חיבור זה מהווה חלק מהדרישות לקבלת
תואר ראשון בהנדסה

מאת :

עילי גם זו לטובה

אוריה אבדר



עזריאלי מכללה אקדמית להנדסה ירושלים
המחלקה להנדסת חשמל ואלקטרוניקה

דימוי מבעד לרקמה מפזרת

חיבור זה מהווה חלק מהדרישות לקבלת
תואר ראשון בהנדסה

מאת :

עילי גם זו לטובה

אוריה אבדר

עבודה זו נעשתה בהדרכתו של ד"ר יוסי דנן מן המחלקה להנדסת חשמל
ואלקטרוניקה של מכללת עזריאלי להנדסה ירושלים.

תוכן עניינים

א.	תקציר
1	פרק 1: מבוא
3	פרק 2: מבנה הרשת Encoder-Decoder
3	2.1 חלוקת הנתונים
3	2.1.1 MaxPooling2D
5	2.1.2 פונקציית אקטיבציה
9	2.2 פעולת הקונבולוציה
9	2.2.1 מטריצת Kernel
12	2.2.2 Batch Normalization
13	2.2.3 פונקציית Loss
13	2.2.4 אופטימיזר Adam
15	פרק 3: יישום מעשי של רשת Encoder-Decoder
15	3.1 חלק ה-Encoder
16	3.2 חלק ה-Decoder
20	פרק 4: ניסוי ראשון – סימולציה
20	4.1 GPU
21	4.1.1 התקנת ה-GPU
22	4.2 Dataset MNIST
23	4.3 הדמיית טשטוש (Blur)
27	פרק 5: ניסוי שני – יישום מעשי של המערכת
27	5.1 RPI Module 4B
28	5.1.1 התקנת מערכת הפעלה ל-RPI
30	5.2 מודל תקשורת UART
31	5.2.1 התקנת הדרייבר ל-UART
31	5.3 מסך ה-Waveshare
33	5.3.1 התקנת תוכנת EPSCOMM
38	5.3.2 הכנת מידע- מעבר לפורמט BMP
40	5.3.3 הצגת התמונות על המסך
45	5.3.4 סגירת משאבים
46	5.3.5 קיבוע למסך ה-Waveshare
47	5.4 מצלמת Basler
49	5.4.1 התקנת תוכנת pylon
50	5.5 תאורה
50	5.6 מהלך הניסוי
50	5.6.1 רקמה מפזרת
53	5.6.2 תוצאות ומסקנות
56	פרק 6: סיכום ומבט לעתיד
57	ביבליוגרפיה
i.	Abstract

רשימת גרפים, איורים, טבלאות

גרף 1 פונקציית ReLU	7
גרף 2 פונקציית Sigmoid	8
גרף 3 model loss	26
גרף 4 model accuracy	26
גרף 5 model loss in practical	54
גרף 6 model accuracy in practical	55
איור 1 הספקטרום האלקטרומגנטי	1
איור 2 מיקום גלי ה-NIR בספקטרום האלקטרומגנטי	2
איור 3 פעולת ה-Max Pooling	4
איור 4 פעולת ה-Skip Connection	5
איור 5 מבנה של נירון מלאכותי	5
איור 6 פעולת מטריצת Kernel	10
איור 7 Batch Normalization	12
איור 8 GPU vs CPU	21
איור 9 בדיקת תקינות ה-GPU	22
איור 10 קטע קוד המבצע טשטוש	23
איור 11 משמאל: התמונות המקוריות, מימין: התמונות המטושטשות	24
איור 12	24
איור 13	25
איור 14	25
איור 15 באיור זה ובאיורים 12, 13 ו-14 ניתן לראות משמאל: את התמונות המטושטשות, ומימין: את פענוח הרשת	25
איור Raspberry Pi Module 4B16	27
איור Raspberry Pi Imager	29
איור 18 מסך הבית של RPI	29
איור CP2102 USB UART HW40919	30
איור Waveshare e-paper screen 4.3inch 800x600	31
איור 21 הורדת שלושת הקבצים והכנסה לתיקייה אחת	34
איור 22 הוספת הקובץ MSCOMM32.OCX לתיקיות SysWOW64 ו-Sys32	34
איור 23 לאחר מתן ההרשאות יש ללחוץ על "אישור"	35
איור 24 תחת התיקייה משלב 2, יש ללחוץ על 'register the control first'	35
איור 25 חלונית המאשרת את ביצוע הפעולה	35
איור 26 תוכנת EPSCOMM	36
איור 27 חלונית תצוגת המסך	36
איור 28 ב-send ניתן לראות את שם התמונה ב-hexa	37
איור 29 תמונת בדיקה של חברת Waveshare	37
איור 30 ביצוע המרה ל-1bpp ללא המרה מקדימה ל-24bpp	38
איור 31 הורדת ה-dataset של MNIST וביצוע resize ל-750x600	38
איור 32 המרת התמונה ל-24bpp	39
איור 33 המרת התמונה ל-1bpp	40
איור 34 פתיחת תקשורת טורית	40
איור 35 הפקודות הקבועות של היצרן להפעלת המסך	41
איור 36 המרת תמונה מ-ASCII ל-hexa	42
איור 37 טבלת ASCII	42
איור 38 חישוב אורך הפקודה	43
איור 39 קוד המראה את מספרי ה-hexa הקבועים ושם תמונה משתנה	43
איור 40 חישוב ה-Parity Byte	43
איור 41 ביצוע XOR	44
איור 42 שליחת הפקודה מקובץ הטקסט למסך	44

45	איור 43 שלבי המעבר ל-hexa
45	איור 44 רשימת הפקודות של image, clear screen ו-refresh
46	איור 45 סגירת המשאבים
47	איור 46 מידות מסך ה-Waveshare
47	איור 47 תכנון רגל ימנית עליונה למסך
48	איור 48 Basler Camera acA1440-220um
48	איור 49 מיקום המצלמה
49	איור 50 מפצל USB 3.0
49	איור 51 תצוגת המסך של תוכנת Pylon
50	איור 52 מימין: תאורת ה-Led, משמאל: תוצאת התמונה המתקבלת לאחר כיוונון התאורה
51	איור 53 מבנה המערכת ללא טשטוש
51	איור 54 מבנה המערכת עם הטשטוש
52	איור 55 התמונות לאחר הדגימה
52	איור 56 התמונות לאחר ביצוע חיתוך ל-200x200
52	איור 57 הכנסת התמונות לרשת כאשר ה-org-וה-blur באותו מיקום
53	איור 58
53	איור 59
53	איור 60
	איור 61 באיור זה ובאיורים 58, 59 ו-60 ניתן לראות בשמאל: את התמונה המטושטשת, ובימין:
54	את פענוח התמונה

33	טבלה 1 חיבור המסך למודל תקשורת UART
41	טבלה 2 פורמט של הפקודות ב-hexa

רשימת קיצורים ור"ת

ע"י על-ידי

ע"פ על-פי

DNA	Deoxyribonucleic Acid
EMR	Electromagnetic Radiation
UV	Ultraviolet
NIR	Near-Infrared Spectroscopy
CAD	Computer-Aided Design
Adam	Adaptive Moment
GPU	Graphics Processing Unit
ASCII	American Standard Code for Information Interchange
UART	Universal Asynchronous Receiver-Transmitter
XOR	Exclusive Or
CNN	Convolutional Neural Network
MSE	Mean Squared Error
RMSProp	Root Mean Square Propagation
RGB	Red, Green, Blue
ML	Machine Learning
RAM	Random-Access Memory
CPU	Central Processing Unit
API	Application Programming Interface
RPI	Raspberry Pi
SD	Secure Digital
DTR	Data Transfer Rate
RXD	Received Data
TXD	Transmit Data
BMP	Bitmap
PIL	Python Imaging Library

תקציר

פרויקט זה בוחן גישה חדשה להדמיה רפואית על-ידי פיתוח מערכת שמטרתה להוות תחליף יעיל לקרני הרנטגן בגלי NIR. צילומי הרנטגן, למרות היותם יעילים מאוד ביצירת תמונות ברורות לאבחון, מהווים סיכונים לבריאות המטופל בשל אורכי הגל הקצרים שלהם והאנרגיה הגבוהה.

גלי NIR, בעלי אורכי גל ארוכים יותר ואנרגיה נמוכה משמעותית, מספקים אלטרנטיבה בטוחה יותר. מאידך, חדירתם המוגבלת גורמת לתמונות להיות מטושטשות, אפקט ההופך אותן לפחות שימושיות עבור אנשי מקצוע רפואיים, עקב קושי בפענוח הממצאים. על-מנת שנוכל להתגבר על אתגר זה, בנינו מערכת אשר תיקח את התמונות המטושטשות ותפענח אותם לתמונות מקוריות וברורות על-ידי רשת Encoder-Decoder.

רשת ה-Encoder-Decoder ברמה התיאורטית, זוהי רשת נוירונים אשר לוקחת מידע מתמונות הקלט ודוחסת אותו למידע קומפקטי, המכונה Feature Vector, על-ידי לקיחת הערכים הדומיננטיים של הפיקסלים. כדי שהמידע לא ילך לאיבוד בדחיסת המידע הוספנו לרשת Skip Connection, המחבר בין כל בלוק של חלק ה-Encoder לבלוק המתאים בחלק ה-Decoder. החלק של ה-Decoder לוקח את ה-Feature Vector ומשחזר אותו לתמונה המקורית.

מערך הנתונים ברשת פוצל לשלוש קבוצות: Train, Validation ו-Test כדי לייעל ולאמת את הביצועים. בכל התהליכים הרשת משתמשת בטכניקות למידה עמוקה סטנדרטיות כגון: פונקציות האקטיבציה ReLU ו-Sigmoid, בפעולות הקונבולוציה, מטריצת Kernel, Batch Normalization, פונקציית Loss ואופטימיזר Adam.

לאחר שכתבנו קוד המיישם את רשת ה-Encoder-Decoder, בנינו סימולציה ממוחשבת למערכת אשר תדמה את הצילומים המטושטשים, וכאשר נכניס את התמונות לרשת הנוירונים נקבל הסרה של הטשטוש.

כדי ליישם זאת באופן מעשי, לקחנו קופסה סגורה לצורך דימוי תנאי מעבדה והנחנו בתוכה משטח אופטי כדי להרכיב עליו את מצלמת ה-Basler, כך שתהיה ב-90° למסך ה-Waveshare. את המסך קיבענו בעזרת רגליים מתאימות אשר הדפסנו במדפסת תלת-מימד. כמו-כן השתמשנו בתאורת Led כדי שהתצלומים יהיו ברורים, ומיקמנו אותה במיקום שימנע חוזר אור בצורה מקסימלית לכיוון המצלמה.

על-מנת שהמערכת תעבוד כראוי, השתמשנו במעבד GPU אשר נועד להתמודד עם משימות של עיבוד תמונה, ב-Raspberry Pi מכיוון שהיינו צריכים מחשב קומפקטי שנוכל להכניס לקופסה ובתקשורת טורית UART בשביל שתהיה תקשורת בין מסך ה-Waveshare ל-Raspberry Pi. בנוסף, על-מנת שיהיה מספיק זרם חשמלי שיוכל להפעיל גם את המצלמה וגם את מסך ה-Waveshare השתמשנו במפצל USB 3.0 שאותו חיברנו ל-Raspberry Pi.

את הניסוי ביצענו על-ידי התמונות של MNIST הנועדו ללמידה עמוקה. לפני ביצוע הניסוי בפועל עשינו סימולציה שתדמה טשטוש על-ידי כתיבת קוד מתאים ולאחר-מכן הכנסנו את התמונות המטושטשות לרשת הנוירונים כדי לראות אם הרשת תצליח להסיר את הטשטוש.

לאחר שראינו כי הניסוי בסימולציה אכן עבד, ביצענו את הניסוי בפועל כאשר את הטשטוש עשינו באמצעות טישו שהנחנו על מסך ה-Waveshare, אשר אמור לדמות רקמה מפזרת בעולם האמיתי והתחלנו לדגום תמונות. לאחר דגימת התמונות דאגנו לסגירת המשאבים לצורך ניהול זיכרון, אבטחה, מניעת התנגשויות וכו'. את התמונות המטושטשות הכנסנו לרשת הנוירונים כדי לראות האם הרשת תצליח לפענח את התמונות.

בניסוי גילינו כי הרשת אכן הצליחה להסיר את הטשטוש בצורה טובה למדיי, הן מבחינת התמונות המפוענחות והן מבחינת הגרפים הממדלים את פונקציית ה-Loss כאשר ניתן לראות מתוכם כי ה-Validation עוקב אחר ה-Train בצורה מדויקת.

מניסוי זה הסקנו כי ביצוע תצלומים למטופלים בעזרת אורכי גל גדולים יותר מקרני הרנטגן, טומנת בחובה הבטחה כאלטרנטיבה בטוחה יותר לקרני הרנטגן באבחון רפואי. יישומים עתידיים פוטנציאליים של טכנולוגיה זו עשויים לכלול גילוי מוקדם של גידולים והתקדמות רפואית אחרת, כאשר טכנולוגיית המערכת ורשת הנוירונים ימשיכו להשתפר.

הדמיה מבוססת NIR יכולה להוות כלי חיוני בטיפול רפואי ברפואה מתקדמת.

הערה: בפרויקט שלנו לא השתמשנו בגלי NIR אלא באורכי גל של מצלמת ה-Basler.

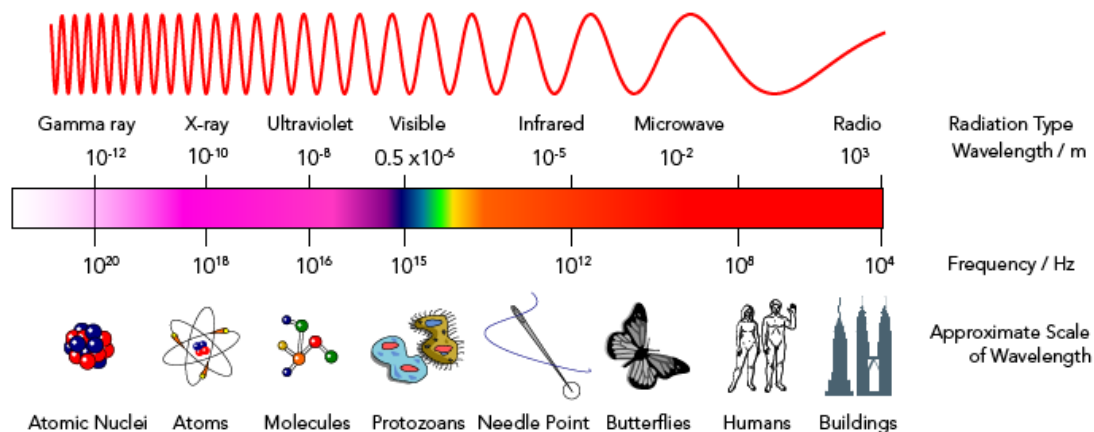
פרק 1: מבוא

גלים הם תנודות יסוד המעבירות אנרגיה למרחקים ארוכים דרך החלל או מדיום, ללא תזוזה פיזית של חלקיקים. גלים אלו מאופיינים באורך גל, תדירות, משרעת ומהירות.

- אורך הגל- הוא המרחק בין שתי פסגות עוקבות של הגל.
- תדירות- זהו מספר המחזורים של הגל בשנייה.
- משרעת- זהו גובה הגל.

ניתן לסווג את הגלים לשני סוגים: גלים מכניים וגלים אלקטרומגנטיים. גלים מכניים אלו גלים הדורשים מדיום (כמו מים, אוויר או מוצקים) כדי להתפשט. לעומת זאת גלים אלקטרומגנטיים, אלו גלים שאינם דורשים מדיום מסוים ויכולים לנוע בוואקום, למשל בחלל.

לאור האמור לעיל, קרינת רנטגן הינה קרינה אלקטרומגנטית מייננת המורכבת מגלים של השדה האלקטרומגנטי, המתפשטים בחלל ונושאים תנע ואנרגיית EMR. בהתאם לתדירות התנודה, נוצרים אורכי גל שונים של ספקטרום אלקטרומגנטי. לרוב קרני הרנטגן יש אורך גל הנע בין 5pm ל-10nm, המקביל לתדרים בטווח של 30petaHz עד 30exaHz ואנרגיות בטווח של 124KeV עד 145KeV. אורכי גל של קרני רנטגן קצרים יותר מאלה של קרני UV וארוכים מאלה של קרני גמא כפי שנראה באיור 1.



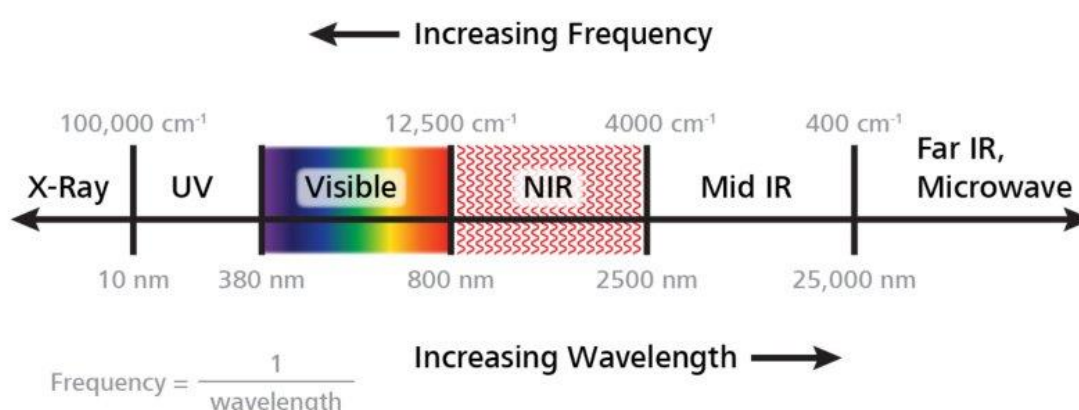
איור 1 הספקטרום האלקטרומגנטי

קרני הרנטגן נקראות כך, על-שם המדען הגרמני וילהלם קונרד רנטגן, שגילה אותם בשנת 1895^[1], במקרה. וילהלם רנטגן, עסק בבחינת שפופרת קתודית במעבדתו. רנטגן החליף את הלוחיות שעל שפופרת קרני הקתודה בנייר שחור ועבה, החשיך את החדר והפעיל את המכשיר. במהלך הניסוי הוא גילה, כי בשעה שהמתח החשמלי בשפופרת מגיע לרמה מסוימת, נפלטת מהשפופרת קרינה בעלת כושר חדירה גדול.

כיום, צילומי הרנטגן הם חלק בלתי נפרד מבתי-חולים ומרכזים רפואיים עכשוויים. אופן הצילום נעשה ע"י סרט צילום הממוקם מאחורי גופו של המטופל. כאשר צילום הרנטגן מופעל, הקרניים

עוברות בקלות דרך העור, ונספגות בעצמות העתירות בסידן^[2]. כתוצאה מכך, לוקח לגלים יותר זמן לחדור דרך העצם, ולכן העצמות נראות הרבה יותר בהירות בצילומי הרנטגן. באמצעות תוצאות צילומי הרנטגן, הרופאים יכולים לפתח תוכניות טיפול יעילות.

כידוע, קרני הרנטגן הינם בעלי אורכי גל קצרים וכתוצאה מכך בצילומי רנטגן אנו נקבל צילומים ברורים. מאידך, האנרגיה של גלים אלו גדולה, דבר היכול להשפיע על בריאות המטופלים. בדרך-כלל, כמות הקרינה אשר נחשפים אליה במהלך צילום רנטגן, שווה ערך לחשיפה של בין מספר ימים לכמה שנים לקרינה טבעית מהסביבה. קרני הרנטגן עלולות לגרום למוטציות ב-DNA, ולכן עלולות להוביל לסרטן בשלב מאוחר יותר בחיים. מסיבה זו, על-מנת שנוכל לקבל תצלומים באופן בטיחותי יותר, נשתמש באורכי גל סביב ה-NIR. NIR אלו גלים הפועלים באזור הקרוב לאינפרא-אדום של הספקטרום האלקטרומגנטי (780nm עד 2500nm)^[3], כפי שנראה באיור 2.



איור 2 מיקום גלי ה-NIR בספקטרום האלקטרומגנטי

בהשוואה לגלי הרנטגן, גלי NIR מאופיינים ברמת חדירה נמוכה ברקמה ובפוטנציאל פיזור גבוה יותר. תכונות אלו, גורמות לאיכות תמונה ירודה המתבטאת ברזולוציה מרחבית נמוכה. כתוצאה מכך, קושי משמעותי נוצר בפענוח הממצאים ובהשגת מידע אבחנתי, בהשוואה לממצאים המתקבלים בבדיקות הרנטגן. לכן אנו נשתמש ברשת Encoder-Decoder שמטרתה תהיה להסיר את הטשטוש שנוצר עקב הרקמה המפזרת וכך נוכל לקבל תמונה שנוכל לפענח.

שימוש בשיטה זו, מאפשר ביצוע בדיקות אבחנתיות תוך הקטנת החשיפה של המטופל לקרינה, ובכך הופך את תהליך האבחון לבטוח יותר. בספר זה נרחיב על רשת זו, כיצד היא פועלת באופן תיאורטי ונציג את היישום המעשי ותוצאות הניסוי.

הערה: בפרויקט שלנו לא השתמשנו בגלי NIR אלא באורכי גל של המצלמה.

פרק 2: מבנה הרשת Encoder-Decoder

כאשר אנו מכניסים מידע לרשת נוירונים לפענוח תמונות מטושטשות, שלב חלוקת הנתונים לשלוש קבוצות נפרדות הינו חשוב להבטחת ביצועים אופטימליים של המודל. חלוקה זו מאפשרת למודל ללמוד את התכונות הרלוונטיות של הנתונים, להעריך את ביצועיו במהלך האימון ולהעריך את יכולתו הכללית להתמודד עם נתונים חדשים שלא נחשף אליהם בעבר.

2.1 חלוקת הנתונים

ראשית נחלק את ה-data שברשותנו לשלושה סטים - Train, Validation ו-Test :

Train (אימון) - סט זה משמש ללמידה של המודל. הרשת נחשפת מספר פעמים (epochs) לנתונים בסט האימון, ומעדכנת את הפרמטרים הפנימיים שלה (המשקלים) בזמן האימון, באמצעות אלגוריתם ההפצה לאחור (Backpropagation). מטרת סט זה היא, למידה של התכונות המאפיינות תמונות מטושטשות ותמונות ברורות, ויצירת מיפוי בין שני סוגי התמונות.

Validation (אימות) - שלב האימות משמש להערכת הביצועים של המודל במהלך האימון עצמו, מבלי שהמודל יתאמן עליו ישירות. מטרת השימוש בסט זה היא לאפשר את כיוון הפרמטרים במודל וקביעת המודל הסופי, למשל מספר השכבות, גודל המודל, קצב הלמידה וכו'. כמו-כן, סט האימות מספק הערכה, האם המודל פשוט מידי למידע (Underfitting) או שהוא מודל מסובך (Overfitting), כך שנוכל לקבוע האם בכלל המודל שיצרנו מתאים למידע שיש ברשותנו או לא.

Test (בדיקה) - שלב הבדיקה משמש להערכת הביצועים הסופיים של המודל לאחר האימון והאימות. סט זה לא נחשף למודל במהלך תהליך האימון או האימות, ולכן מאפשר הערכה אובייקטיבית של היכולת הכללית של המודל להתמודד עם מידע חדש ולא מוכר. הביצועים של סט הבדיקה נותנים הערכה טובה עד כמה המודל יהיה שימושי בפועל בעולם האמיתי.

בדרך-כלל, הנתונים מחולקים באופן הבא :

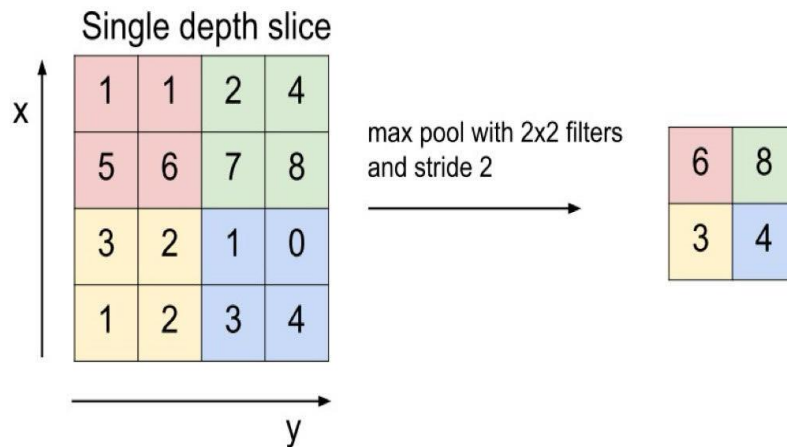
- Train - מהווה כ-70% מכלל הנתונים.
- Validation - מהווה כ-10% מכלל הנתונים.
- Test - מהווה כ-20% מכלל הנתונים.

2.1.1 MaxPooling2D

אנו נחלק את ההסבר לשני חלקים: שלב ה-Encoder ושלב ה-Decoder. החלק של ה-Encoder ברשת, מבצע Max Pooling על התמונה בקלט ע"י דחיסה של המידע שלה לייצוג קומפקטי, המכונה גם Feature Vector. למשל, אם ישנה מטריצה בגודל 4x4 נשנה את גודלה למטריצת 2x2 ע"י כך שבכל תת-מטריצה 2x2 מהמטריצה המקורית, אנו לוקחים את הערך הגבוה ביותר^[4].

כלומר, ההקטנה לוקחת את הערכים הכי דומיננטיים בתמונה וכך ייווצר לנו ה-Feature Vector כפי שניתן לראות באיור 3.

MAX POOLING



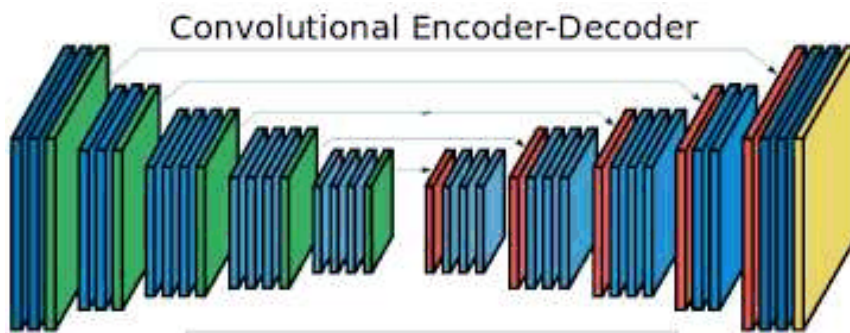
איור 3 פעולת ה-Max Pooling

ל-Max Pooling ישנם מספר יתרונות:

- **שמירה על תכונות חשובות**- ההנחה שהערך המקסימלי במאגר מקומי הוא תכונה חזקה. ע"י שמירה על ערכי מקסימום אלו אנו נשמור על התכונות החשובות כמו קצוות או טקסטורות.
- **מפחית עומס חישובי**- גדלי קלט קטנים יותר פירושם פחות פרמטרים לעיבוד בשכבות הבאות, מה שמאיץ את החישוב והופך את הרשת ליעילה יותר.

נשים לב, כי כאשר אנו מבצעים את ה-Encoder, מתבצע תהליך של Down Sampling. כלומר, נוצר מצב של איבוד מידע. אי-לכך על-אף שאנו מקבלים את התכונות החשובות שבתמונה, עדיין יש מידע שנאבד במעבר בין השכבות. לכן אנו נוסיף לרשת Skip Connection, שמטרתו תהיה לשמור על המידע ולחבר בין כל בלוק בשכבת ה-Encoder לבלוק שמתאים לו בשכבת ה-Decoder וכך אנו נשמור על המידע מבלי שהוא יאבד.

את אופן הפעולה של ה-Skip Connection ניתן לראות באיור 4.



איור 4 פעולת ה-Skip Connection

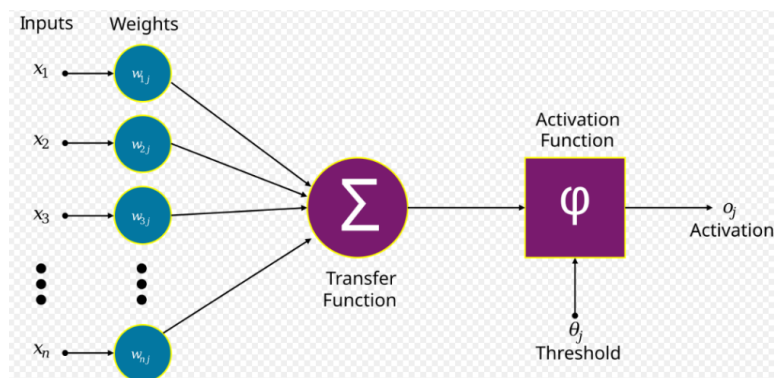
החלק של ה-Decoder ברשת, מקבל את ה-Feature Vector מהחלק של ה-Encoder ומשחזר אותו לתמונה המקורית.

2.1.2 פונקציית אקטיבציה

פונקציית אקטיבציה של נוירון ברשת עצבית מלאכותית, היא פונקציה מתמטית המחשבת את הפלט של הנוירון ביחס לקלטיו שהוא מקבל ולמשקלם. פונקציות האקטיבציה הן המרכיב המרכזי ברשתות נוירונים [5]. לפונקציה הנבחרת יש השפעה על ביצועי הרשת, ועל ההתאמה שלו לביצוע המשימה שעליה אומנה. מטרת הפונקציה היא לקבוע את עוצמת הפלט של הנוירון, ביחס לקלט שקיבל. ההשראה לאופן הפעולה של פונקציית אקטיבציה התקבלה מאופן הפעולה של תאי העצב במוח, אשר הפלט שלהם נקבע בהתאם לפוטנציאל הפעולה.

נוירון ברשת נוירונים מלאכותית פועל כ-Perceptron (אלגוריתם בסיסי בתחום למידת המכונה), כך שהוא מקבל וקטור קלט (למשל תמונה), ומכפיל אותו בווקטור משקולות שנלמד בזמן האימון. לאחר-מכן, פלט הנוירון נקבע באמצעות העברת תוצאת המכפלה בפונקציית האקטיבציה.

ניתן לראות למשל באיור 5 מבנה של נוירון מלאכותי, המציג את המיקום של פונקציית האקטיבציה לאחר הפעלת המשקלים ולפני הפלט :



איור 5 מבנה של נוירון מלאכותי

מטרה נוספת של פונקציית האקטיבציה היא להכניס אי-לינאריות לרשת, ולכן הפונקציות הנמצאות בשימוש רווח הן פונקציות לא לינאריות, או פונקציות שיש בהן אלמנט כלשהו של אי-לינאריות. פונקציית אקטיבציה לא לינארית מאפשרת לפתור בעיות לא טריוויאליות באמצעות חיבור של מספר קטן של צמתים.

ישנם ארבעה פונקציות אקטיבציה נפוצות:

1. **ReLU (Rectified Linear Unit)** - זוהי הפונקציה הנפוצה ביותר. פונקציה זו פשוטה לחישוב ומאפשרת לרשת ללמוד תכונות מורכבות בצורה יעילה.
 2. **Sigmoid** - פונקציה זו, מפיקה פלט בטווח בין 0 ל-1 והיא שימושית למשימות סיווג בינארי.
 3. **Tanh (Hyperbolic Tangent)** - פונקציה זו, מפיקה פלט בטווח בין -1 ל-1 והיא דומה ל-Sigmoid, אך סימטרית סביב האפס.
 4. **SoftMax** - פונקציה זו, משמשת בעיקר בשכבת הפלט של רשתות סיווג רב-מחלקתי, והיא ממפה את הווקטור של הקלטים להסתברויות על-פני כל המחלקות.
- ברשת המוצגת בפרויקט זה, השתמשנו בפונקציית Sigmoid ו-ReLU ועליהן נרחיב.

פונקציית ReLU

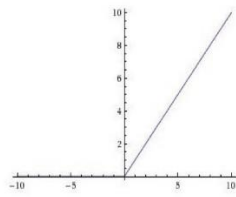
פונקציית ReLU (Rectified Linear Unit), היא אחת מפונקציות ההפעלה הנפוצות ביותר בלמידה עמוקה, במיוחד ברשתות נוירונים קונבולוציוניות (CNNs). הפופולריות שלו נובעת מהפשטות והיעילות שלו.

פונקציית ReLU מזכירה מאוד את פונקציית ה-Ramp, כאשר:

$$\begin{aligned} ReLU(x) &= x, & x > 0 \\ ReLU(x) &= 0, & x \leq 0 \end{aligned} \quad (2.1)$$

בגרף 1 נשים לב כי כאשר הערכים הם שליליים נקבל 0, וכאשר הערכים חיוביים נקבל קו לינארי.

Activation Functions



ReLU
(Rectified Linear Unit)

- Computes $f(x) = \max(0, x)$

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

[Krizhevsky et al., 2012]

גרף 1 פונקציית ReLU

למרות שפונקציית ReLU נראית פשוטה, היא מציגה אי-לינאריות לרשת הנוירונים. תכונה זו חיונית מכיוון שללא פונקציות הפעלה לא-לינאריות, רשת הנוירונים תהיה שוות ערך ל-Perceptron חד-שכבתי, ללא קשר למספר השכבות ולא תוכל ליצור מודל של פונקציות מרוכבות.

לפונקציית ReLU ישנן מספר תכונות חשובות:

- פונקציית ReLU פשוטה ומהירה מאוד לחישוב, מה שמאיץ את תהליך האימון של הרשת.
- בניגוד לפונקציות אקטיבציה כ-Tanh ו-Sigmoid, ReLU אינה סובלת מדעיכה חזקה של הגרדיאנט. לכן זה מאפשר לרשתות נוירונים עמוקות להתאמן בצורה יעילה יותר.
- ReLU מוסיפה אלמנט של אי-לינאריות למודל, מה שמאפשר לרשתות נוירונים ללמוד קשרים מורכבים בין הקלט לפלט^[6].

עם-זאת ישנן שתי בעיות פוטנציאליות עם ReLU:

1. נוירונים עלולים שלא לעבוד כראוי במהלך האימון אם הם נתקעים באזור השלילי של הקלט, מכיוון שהשיפועים שלהם הופכים לאפס והם מפסיקים להתעדכן.
2. מכיוון ש-ReLU אינו מוגבל בצד החיובי, הפלט יכול להיות גדול מאוד, מה שעלול לגרום לאי-יציבות מספרית ברשת.

כדי לטפל בחלק מהמגבלות שלו, פותחו כמה גרסאות של ReLU:

LReLU (Leaky ReLU) - פונקציה זו מציגה שיפוע קטן עבור קלט שלילי^[7] במקום להגדיר אותו לאפס. תכונה זו עוזרת להפחית את בעיית הנוירונים המתים. היא מוגדרת כך:

$$(2.2) \quad f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

כאשר α הוא קבוע חיובי קטן (למשל 0.01).

PReLU (Parametric ReLU) - דומה ל-Leaky ReLU, אך השיפוע עבור קלט שלילי α נלמד במהלך האימון^[8]. תכונה זו נותנת יותר גמישות להסתגל לנתונים.

פונקציית ReLU הפכה לפופולרית בתחום למידת מכונה בשנים 2010-2012, בעיקר בזכות חוקרים כמו Geoffrey Hinton ו-Yan LeCun. אחת העבודות המוקדמות שהציגה את השימוש הנרחב של ReLU הייתה במחקר Vinod Nair ו-Geoffrey E. Hinton בשנת 2010, במאמר בשם: "Rectified Linear Units Improve Restricted Boltzmann Machines". ReLU זכתה לפופולריות רבה בשנים שלאחר-מכן והפכה לפונקציית האקטיבציה המועדפת ברוב רשתות הנוירונים העמוקות.

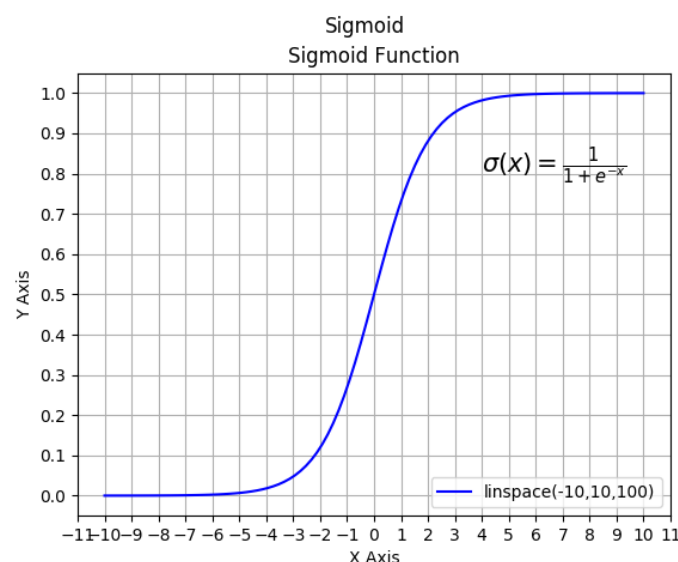
פונקציית Sigmoid

פונקציית ה-Sigmoid, היא אחת מפונקציות האקטיבציה הוותיקות והנפוצות ביותר בשימוש ברשתות נוירונים. ההיסטוריה שלה חוזרת לשימושים מוקדמים בתחום הסטטיסטיקה והביולוגיה, אך בתחום של למידת מכונה היא נעשתה פופולרית במיוחד במהלך שנות ה-80 וה-90.

פונקציית Sigmoid מוגדרת ע"י המשוואה הבאה:

$$(2.3) \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$

הפלט של הפונקציה יהיה תמיד בטווח שבין 0 ל-1. פלט זה הופך את הפונקציה לשימושית, במיוחד עבור מודלים שבהם הפלט צריך לייצג הסתברות. לפונקציית ה-Sigmoid יש עקומה בצורת S כפי שניתן לראות בגרף 2, וזו הסיבה שהיא מכונה לעיתים קרובות "עקומת S" [9]. היא עוברת בצורה חלקה מ-0 ל-1, מה שהופך אותה לשימושית עבור בעיות סיווג בינארי.



גרף 2 פונקציית Sigmoid

הנגזרת של פונקציית ה-Sigmoid היא :

$$(2.4) \quad \sigma'(x) = \sigma(x) * (1 - \sigma(x))$$

נגזרת זו חשובה לתהליך ההפצה לאחור ברשתות נוירונים, שכן היא מאפשרת לרשת להתאים את המשקולות בזמן האימון.

בנוסף, אנו נשים את פונקציית ה-Sigmoid בסוף הרשת מכיוון שהיא מדמה את פעולת ה-Perceptron במוח.

2.2 פעולת הקונבולוציה

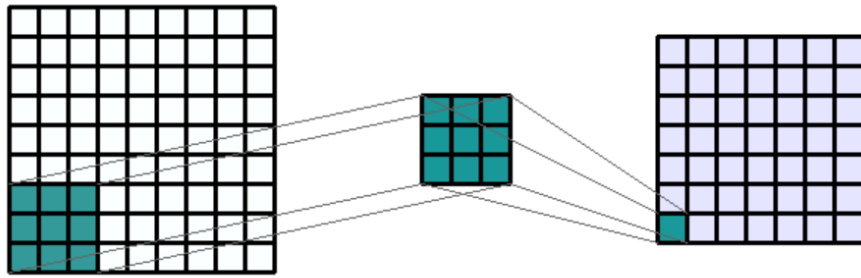
פעולת הקונבולוציה היא פעולה בינארית בין שתי פונקציות או סדרות ערכים. יש לה שימושים בהתמרות אינטגרליות כדוגמת התמרת פורייה, התמרת לפלס, עיבוד אותות, בסטטיסטיקה ובתחומים נוספים במתמטיקה, פיסיקה והנדסה.

פעולת הקונבולוציה היא הבסיס לפעולת הרשתות הנוירוניות הקונבולוציוניות (CNNs), המשמשות בעיקר לעיבוד תמונות. פעולה זו מאפשרת "לראות" תכונות מקומיות בתמונה כגון: קווים, צורות וטקסטורות וללמוד אותן בצורה יעילה.

2.2.1 מטריצת Kernel

מטריצת Kernel, או מסנן, היא מערך דו-ממדי קטן המשמש בפעולת הקונבולוציה ברשתות נוירוניות קונבולוציוניות, לזיהוי תכונות מקומיות מנתוני הקלט. מטריצת Kernel מחליקה על-פני התמונה משמאל לימין, מלמעלה ומלמטה ע"י יישום פעולת הקונבולוציה, תוך כדי ביצוע מכפלה נקודתית בין הערכים שלה לבין הערכים בתמונה. תוצאות מכפלות הנקודה בכל נקודה, מסוכמות ויוצרות ערך יחיד. התהליך חוזר על עצמו עבור כל נקודה ונקודה בתמונה, ויוצר מפה חדשה של תכונות. הפלט של פעולה זו נקרא תמונה מסוננת.

לצורך המחשת הפעולה ניקח מטריצת Kernel בגודל 3×3 , המסנן תמונה בגודל 9×9 כפי שנראה באיור 6 המסנן נע על-פני כל התמונה כדי ללכוד בתמונה את כל הריבועים באותו גודל (3×3). מכפלת הקונבולוציה היא כפל אלמנטי, והסכום של תוצאה זו הוא הפיקסל המתקבל בתמונת הפלט (או המסוננת).



איור 6 פעולת מטריצת Kernel

בנוסף, אפשר לבחור שלא יהיו מטריצות Kernel בגדלים מרובעים. ישנה אפשרות להחליט על מטריצות Kernel בגבהים וברחבים שונים. זה קורה לעיתים קרובות בניתוח תמונות ואותות. למשל, כאשר נרצה לסרוק את התמונה של אות או צליל, גודל המטריצה המועדפת תהיה 1x5.

ברשת ה-Encoder-Decoder, kernel משמש בשכבות הקונבולוציה כדי לדחוס את נתוני הקלט לייצוג מופשט יותר ולאחר-מכן לשחזר את התמונה עם טשטוש מופחת הנגרם ע"י רקמה מפזרת. הבחירה והלמידה של מטריצות Kernel חיוניות כדי להסיר ביעילות את הטשטוש ולשחזר את המבנה הבסיסי בתמונות.

מטריצת Kernel מיוצגת ע"י הנוסחה הבאה (2.5):

$$(2.5) \quad \omega * F(x, y) = \left(\sum_{\delta x = -k_i}^{k_i} \sum_{\delta y = -k_j}^{k_j} \omega(\delta x, \delta y) F(x + \delta x, y + \delta y) \right) + \omega_{bias}$$

להלן יוצג הסבר מפורט על הנוסחה (2.5):

- $F(x, y)$ - זהו הקלט (במקרה שלנו זאת תמונה), כאשר x ו- y מציינים את מיקומי הפיקסלים בקלט.
- $\omega(\delta x, \delta y)$ - זוהי ליבת הקונבולוציה שהוחל על הקלט. לליבה זו יש ממדים התואמים את מיקומי הפיקסלים של הקלט סביב הנקודה x ו- y .
- הסיגמות $(\sum_{\delta x = -k_i}^{k_i} \sum_{\delta y = -k_j}^{k_j} \dots)$, מייצגות את הסכום על כל חלון הליבה סביב כל נקודה של הקלט.

- ω_{bias} - זוהי הסטייה שהתווספה לתוצאה של הקונבולוציה. ברשתות נוירוניות הסטייה עוזרת לשנות את הפלט של פעולת הקונבולוציה, ומוסיפה גמישות ליכולת של המודל להתאים נתונים. בפעולת המוח תמיד ישנו מתח קבוע ולכן נצטרך להוסיף אותו.
- תוצאת הקונבולוציה היא החלת הליבה על מפת תכונת הקלט והוספת הסטייה.

על-מנת שנוכל לחשב את גודל התמונה לאחר פעולת הקונבולוציה נשתמש בנוסחאות הבאות (2.6):

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$(2.6) \quad H_2 = \frac{H_1 - F + 2P}{S} + 1$$

$$D_2 = K$$

להלן יוצג הסבר מפורט על הנוסחאות (2.6):

- $W_1 * H_1 * D_1$ - זהו נפח הקלט כאשר: W_1 מייצג את מספר השורות במטריצה, H_1 מייצג את מספר העמודות ו- D_1 מייצג את עומק הקלט.
 - השכבה הקונבולוציונית משתמשת בארבעה היפר-פרמטרים מרכזיים:
 - $-K$ מספר הפילטרים
 - $-F$ היקף או גודל מרחבי של הפילטר
 - $-S$ זהו גודל הקפיצות שבו הפילטר מוחל
 - $-P$ כמות ריפוד האפסים שהוחלה על נפח הקלט. הריפוד משמש לשליטה בגודל נפח הפלט ולשמירה על הממדים המרחביים של הקלט
- ניקח לצורך הדוגמה מטריצה בגודל (32,32,3), כאשר מספר הפילטרים יהיה $K = 6$ וגודל המטריצה של הפילטר יהיה 5x5. לאחר החישוב נקבל את המידות החדשות:

$$W_2 = \frac{32 - 5 + 0}{1} + 1 = 28$$

$$(2.7) \quad H_2 = \frac{32 - 5 + 0}{1} + 1 = 28$$

$$D_2 = K = 6$$

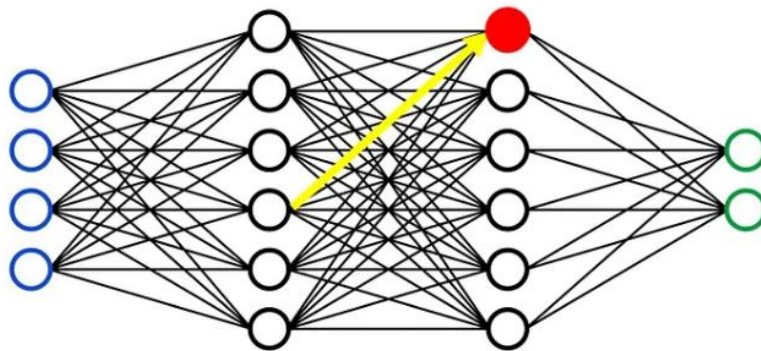
ולכן התמונה שלנו תשנה את ממדיה מ-(32,32,3) ל-(28,28,6).

Batch Normalization 2.2.2

ברשתות נוירונים עמוקות, כאשר נתוני הקלט עוברים דרך שכבות מרובות, התפלגות הנתונים יכולה להשתנות. תופעה זו, הידועה בתור הסטת משתנים פנימיים^[10], יכולה להאט את תהליך האימון ולהקשות על בחירת היפר-פרמטרים טובים.

Batch Normalization מטפל בבעיה זו על-ידי נרמול ההפעלה של כל שכבה בנפרד. פעולת הנרמול לוקחת את הערכים ומחזירה אותם לתחום החיובי והשלילי בכדי שפונקציית האקטיבציה ReLU תוכל לעבוד כראוי. במידה ולא היינו עושים את פעולת ה-Batch Normalization, הערכים היו נשארים חיוביים, מה שגורם לחוסר יציבות במערכת, ובעקבות כך פונקציית ה-ReLU לא הייתה פועלת כראוי.

Batch normalization



איור 7 Batch Normalization

נרמול הנתונים נעשה ע"י הנוסחה הבאה (2.8):

$$(2.8) \quad \hat{x}^{(K)} = \frac{x^{(K)} - E[x^{(K)}]}{\sqrt{\text{Var}[x^{(K)}]}}$$

בנוסחה זו, אנו מפחיתים את הממוצע של מערך הנתונים מכל נקודת נתונים ומחלקים את ההפרש בסטיית התקן של מערכי הנתונים. פעולה זו מאלצת את הנתונים הסטנדרטים לקבל ממוצע של אפס וסטיית תקן של 1.

יתרונות של Batch Normalization:

- **אימון מהיר יותר** - מפחיתה את מספר האיטרציות באימון כדי להגיע לרמת ביצועים מסוימת.
- **אפקט הרגוליזציה** - פועלת כצורה של רגוליזציה ומפחיתה את הצורך בטכניקות כמו dropout.

- **זרימת שיפוע יציבה** - מפחיתה את בעיות השיפוע הנעלמות או המתפוצצות, מה שמקל על אימון הרשת.
- **רגישות מופחתת לאתחול** - הופכת רשתות נוירונים לפחות תלויות בבחירת המשקלים הראשוניים.

2.2.3 פונקציית Loss

פונקציית Loss (או פונקציית ההפסד), היא מדד המשמש בלמידה עמוקה כדי להעריך עד כמה המודל החזוי מתואם לערכים האמיתיים^[11]. פונקציית Loss מחושבת ע"י השוואה בין התוצאות החזויות של המודל לבין הערכים האמיתיים, ומטרתה לספק מדד לפער או לשגיאה. מטרת המודל היא למזער את פונקציית ה-Loss במהלך האימון כדי לשפר את הדיוק והביצועים שלו.

ברשת שלנו אנו נשתמש בפונקציית Loss מסוג MSE שנוסחתה היא :

$$MSE = \frac{1}{N} \sum_{i=0}^N (Y_i - \hat{Y}_i)^2 \quad (2.9)$$

כאשר :

- y_i - הוא הערך האמיתי
- y^i - הוא הערך החזוי
- N - הוא מספר הדגימות.

מכיוון שנרצה לשחזר את התמונה ולהקטין את הרעשים, אנו נצטרך להשוות בין ערכי הפיקסלים של התמונה המקורית והתמונה המשוחזרת. בחירה זו של פונקציית MSE תיתן לנו את ההשוואה ביניהם. בנוסף, ישנו פרמטר של קצב הלמידה המשפיעה על ההפסד הנקרא Learning Rate. מטרתו היא לקבוע את גודל "הצעד" שהמודל ייקח בכל עדכון פרמטרים, במהלך תהליך האימון. במילים אחרות, הוא משפיע על האופן שבו המודל מתאים את המשקולות שלו כדי למזער את פונקציית ה-Loss.

2.2.4 אופטימיזר Adam

אופטימיזר Adam (Adaptive Moment Estimation), הוא אחד מאלגוריתמי האופטימיזציה הפופולריים ביותר המשמשים באימון רשתות נוירוניות. אופטימיזר Adam מתבסס על החזקות של שתי טכניקות פופולריות אחרות: AdaGrad ו-RMSProp. מתאים את קצב הלמידה עבור כל פרמטר בנפרד בהתבסס על שיפועי עבר. כלומר, יקטין את קצב הלמידה עבור שיפועים גדולים ומגדיל עבור פרמטרים עם שיפועים קטנים. RMSProp עושה ממוצע משוקלל בין הנגזרות שהיו בעבר לנוכחית.

אופטימיזר Adam קובע כיצד לעדכן את הפרמטרים ביעילות כדי להגיע למינימום של פונקציית הפסד. הוא מיישם אלגוריתמים כדי לנווט במרחב הפרמטרים, במטרה למצוא את הפרמטרים האופטימליים שגורמים לאובדן הנמוך ביותר האפשרי.

איך אופטימיזר Adam עובד?

Adam עוקב אחר ממוצע דעיכה אקספוננציאלית של שיפועי עבר^[12] (m_t). הוא מייצג את הממוצע של השיפוע, ועוזר למיטוב לעקוב אחר כיוון הנתיב של השיפוע.

הנוסחה ל-Adam היא (2.10):

$$(2.10) \quad m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

כאשר β_1 הוא קצב הדעיכה ו- g_t הוא גרדיאנט ההפסד ביחס לפרמטרים בשלב הזמן t . כלי זה מאפשר ניהול חלק יותר של עדכונים תכופים ולנווט בשיפועים רועשים.

Adam גם מבצע חישוב של ממוצע דעיכה אקספוננציאלית של ריבועי שיפועי עבר v_t . החישוב עוזר לשנות את קצב הלמידה עבור כל פרמטר, כך שמאפשר עדכונים גדולים יותר עבור תכונות נדירות ועדכונים קטנים עבור תכונות תכופות.

החישוב מיוצג ע"י הנוסחה הבאה (2.11):

$$(2.11) \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

באיטרציות הראשוניות, הממוצעים שחושבו ע"י Adam עשויים להיות מוטים לאפס. כדי לנטרל את עובדה זו, Adam כולל מונחי תיקון הטיה (2.12):

$$(2.12) \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad ; \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

הפרמטרים של הרשת הנוירונית מתעדכנים באמצעות הכלל הבא (2.13):

$$(2.13) \quad \theta_{t+1} = \theta_t - \frac{\alpha * \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- ϵ - הוא קבוע קטן שנוסף ליציבות מספרית
- θ_t - מייצג את הפרמטרים בשלב הזמן t
- α - זהו קצב הלמידה

פרק 3: יישום מעשי של רשת Encoder-Decoder

לאחר שדנו בבסיס התיאורטי של רשת ה-Encoder-Decoder, כעת נעבור ליישומה המעשי. בפרק זה נפרט את היישום של ארכיטקטורת Encoder-Decoder באמצעות סביבת העבודה PyCharm, עם התמקדות מיוחדת בספריות TensorFlow ו-Keras.

3.1 חלק ה-Encoder

בחלק זה יוצג כיצד המידע עובר בחלקו של רשת ה-Encoder ופירוט כל שכבה:

בלוק מספר 1:

- ראשית התמונה עוברת דרך conv2D כאשר יש לה 64 ניוונים, מטריצת kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 64 ניוונים, מטריצת kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה תעבור דרך שכבת MaxPooling2D כאשר גודל מטריצת Kernel היא 2×2 .
- לבסוף נעביר דרך שכבת ה-BatchNormalization.

בלוק מספר 2:

- ראשית התמונה עוברת דרך conv2D כאשר יש לה 128 ניוונים, מטריצת kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 128 ניוונים, מטריצת kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה תעבור דרך שכבת MaxPooling2D כאשר גודל מטריצת Kernel היא 2×2 .
- לבסוף נעביר דרך שכבת ה-BatchNormalization.

בלוק מספר 3:

- התמונה עוברת דרך conv2D כאשר יש לה 256 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 256 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU עם padding מסוג valid.
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.

- לאחר-מכן התמונה תעבור דרך שכבת MaxPooling2D, כאשר גודל מטריצת Kernel היא 2×2 עם padding מסוג valid.

בלוק מספר 4:

- התמונה עוברת דרך conv2D כאשר יש לה 512 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 512 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU עם padding מסוג valid.
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן התמונה תעבור דרך שכבת MaxPooling2D, כאשר גודל מטריצת Kernel היא 2×2 עם padding מסוג valid.
- לבסוף נעביר דרך שכבת ה-BatchNormalization.

בלוק מספר 5:

- התמונה עוברת דרך conv2D כאשר יש לה 1024 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 1024 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU עם padding מסוג valid.
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן התמונה תעבור דרך שכבת MaxPooling2D, כאשר גודל מטריצת Kernel היא 2×2 עם padding מסוג valid.
- לבסוף נעביר דרך שכבת ה-BatchNormalization.

בסוף הבלוק החמישי נעביר שוב את הפלט דרך שכבת ה-BatchNormalization ונקבל את הערך המוצפן.

3.2 חלק ה-Decoder

לאחר שלב ה-Encoder אנו נכניס את הערכים הדומיננטיים של התמונה ונעביר אותה לחלק השני של הרשת הנקרא Decoder.

בלוק מספר 6:

- תוצאת ה-Encoder עוברת דרך conv2D כאשר יש לה 1024 פילטרים, מטריצת kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 1024 פילטרים, מטריצת kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.

- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן נעביר דרך ה-skip connection כך שנחבר את בלוק מספר 5 לבלוק מספר 6 (וכך המידע ישמר).
- לאחר-מכן התמונה תעבור דרך שכבת UpSampling2D כאשר גודל מטריצת Kernel היא 2×2 .
- לבסוף נעביר דרך שכבת ה-BatchNormalization.

בלוק מספר 7:

- תוצאת בלוק 6 עוברת דרך conv2D כאשר יש לה 512 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 512 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן התמונה תעבור דרך שכבת UpSampling2D כאשר גודל מטריצת Kernel היא 2×2 .
- לאחר-מכן נעביר דרך Skip Connection כך שנחבר את בלוק 4 לבלוק 7 (וכך ישמר המידע).

בלוק מספר 8:

- תוצאת בלוק 7 עוברת דרך conv2D כאשר יש לה 256 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 256 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן התמונה תעבור דרך שכבת UpSampling2D כאשר גודל מטריצת Kernel היא 2×2 .
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן נעביר דרך Skip Connection כך שנחבר את בלוק 3 לבלוק 8 (וכך ישמר המידע).

בלוק מספר 9:

- תוצאת בלוק 8 עוברת דרך conv2D כאשר יש לה 128 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן נרפד באפסים בגודל 1×1 .

- לאחר-מכן התמונה תעבור דרך שכבת UpSampling2D כאשר גודל מטריצת Kernel היא 2×2 .
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 128 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן נעביר דרך Skip Connection כך שנחבר את בלוק 2 לבלוק 9 (וכך ישמר המידע).

בלוק מספר 10:

- תוצאת בלוק 9 עוברת דרך conv2D כאשר יש לה 64 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 64 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן התמונה תעבור דרך שכבת UpSampling2D כאשר גודל מטריצת Kernel היא 2×2 .
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן נעביר דרך Skip Connection כך שנחבר את בלוק 1 לבלוק 10 (וכך ישמר המידע).

לבסוף נעביר בשכבה נוספת (שקולה ל-fully connected) שנתוניה הן :

- תוצאת בלוק 10 עוברת דרך conv2D כאשר יש לה 32 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן התמונה עוברת דרך conv2D נוספת כאשר יש לה 32 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג ReLU.
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לאחר-מכן התמונה תעבור דרך שכבת UpSampling2D כאשר גודל מטריצת Kernel היא 2×2 .
- לאחר-מכן נעביר דרך שכבת ה-BatchNormalization.
- לבסוף נעביר דרך conv2D כאשר יש לה 3 פילטרים, מטריצת Kernel בגודל 3×3 ועם פונקציית אקטיבציה מסוג Sigmoid שבה נקבל תמונה בעלת 3 ערוצים (RGB) כמו הקלט.

לאחר שהבנו את מבנה הרשת, נוכל לבנות את המודל ולאמן אותו :

בניית המודל:

- ראשית ניצור מודל כאשר :
 - שכבת הקלט שלו תהיה התמונה
 - שכבת הפלט שלו תהיה ה-decode
- נייצר פונקציית Loss : MSE
- נגדיר שהמודל יעבוד תחת אופטימיזציה של Adam עם פונקציית ה-Loss שהגדרנו
- נשמור את המודלים ב-checkpoints (מהם נבחר את ה-epoch הכי טוב של המודל)

אימון המודל:

אנו נאמן את המודל לפי הפרמטרים הבאים :

- נחלק את המידע ל-batch כדי שניהול הזיכרון יהיה יעיל
 - נשתמש בנתונים הן כקלט והן כפלט
 - נגדיר את מספר ה-epochs להיות 100 (כמות הפעמים שנעבור על המידע)
 - נגדיר את מספר ה-batch ל-25
- לבסוף נעביר תמונות שהרשת לא ראתה בשלב האימון והאימות ונראה האם היא תצליח לחזות את התמונות המקוריות.

פרק 4: ניסוי ראשון – סימולציה

לאחר ההסבר היסודי של מבנה הרשת Encoder-Decoder, הן מנקודת מבט תיאורטית והן באמצעות יישום אלגוריתמי ע"י שימוש בקוד בסביבת העבודה PyCharm, עלינו לוודא כי הרשת עובדת כראוי. השלב הראשון שעלינו לעשות לפני שנתחיל בניסוי הוא, להתקין ולהגדיר GPU במערכת שלנו. שלב זה חיוני כדי להגדיל את כוח החישוב של המערכת ולאפשר אימון ועיבוד יעילים יותר של הרשת.

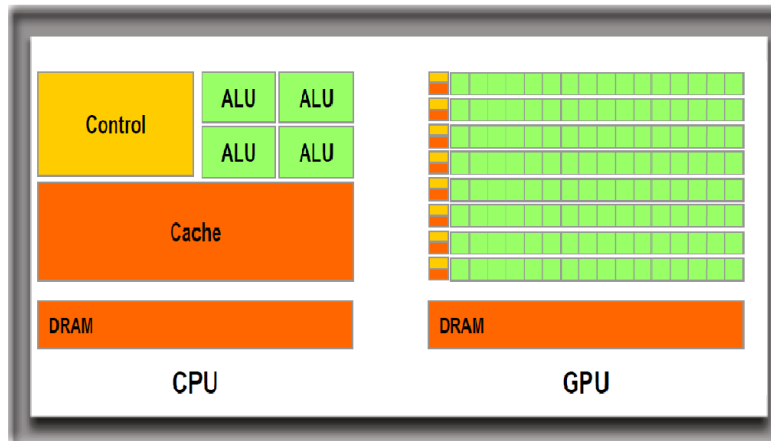
GPU 4.1

GPU (Graphics Processing Unit), זהו למעשה מעגל אלקטרוני שיכול לבצע חישובים מתמטיים במהירות גבוהה. משימות מחשוב כמו עיבוד גרפי, למידת מכונה (ML) ועריכת וידאו, דורשות יישום של פעולות מתמטיות דומות על מערך נתונים גדול. העיצוב של ה-GPU מאפשר לו לבצע את אותה פעולה על מספר ערכי נתונים במקביל, וכך זה מגדיל את יעילות העיבוד שלו עבור משימות עתירות מחשוב רבות.

מעבדי GPU's פועלים באמצעות שיטה הנקראת עיבוד מקבילי, כאשר מספר מעבדים מטפלים בחלקים נפרדים של משימה אחת. ל-GPU יש זיכרון RAM משלו לאחסון הנתונים שהוא מעבד. זיכרון RAM זה תוכנן במיוחד, כדי להחזיק את כמויות המידע הגדולות הנכנסות ל-GPU עבור מקרי שימוש אינטנסיביים ביותר בגרפיקה.

עבור יישומי גרפיקה, ה-CPU וה-GPU פועלים יחד כדי להגדיל את תפוקת הנתונים ואת מספר החישובים במקביל בתוך אפליקציה. במילים אחרות, ה-GPU משלים את ארכיטקטורת ה-CPU בכך שהוא מאפשר לחישובים חוזרים בתוך האפליקציה להיות מופעלים במקביל בזמן שהתוכנית הראשית ממשיכה לרוץ על ה-CPU.

ההבדל העיקרי בין ה-GPU ל-CPU הוא, שה-CPU נועד להתמודד במהירות עם מגוון רחב של משימות, החל מחישובים פשוטים ועד להפעלת תוכנות מורכבות אך הוא פחות יעיל במשימות הדורשות חישובים רבים במקביל. GPU לעומת-זאת, תוכנן להתמודד עם חישובים גרפיים רבים במקביל. הוא מחלק משימות רבות לחלקים קטנים ומבצע אותם בו-זמנית, מה שהופך אותו ליעיל מאוד עבור עיבוד תמונה ולמידת מכונה.



איור 8 GPU vs CPU

בשנים האחרונות הגישה ל-GPU התחילה להיות נגישה גם דרך הענן, דבר שהופיע כחלופה לפריסות GPU מסורתיות. GPU בענן, הוא שירות GPU וירטואלי המסיר את הצורך בפריסת GPU או חומרה ותוכנה קשורים במכשיר מקומי. משתמשים יכולים לגשת למעבדי GPU's בענן ע"פ דרישה מדפדפן אינטרנט (למשל, Google Collab) עבור מגוון רחב של יישומים כולל עיבוד תלת-מימד, למידת מכונה, ניתוח נתונים ועוד.

4.1.1 התקנת ה-GPU

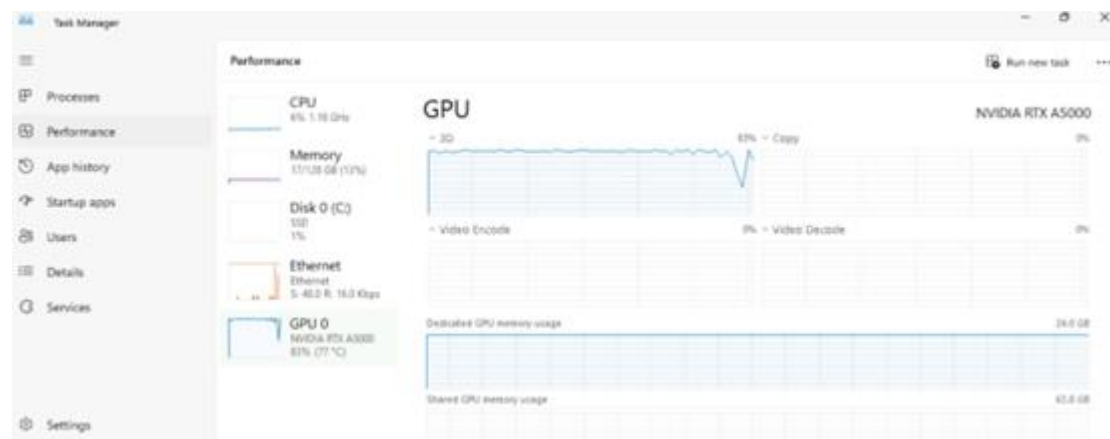
בפרויקט שלנו השתמשנו ב-GPU מסוג RTX A5000 של חברת Nvidia. ב-GPU מסוג זה, על-מנת שנוכל לקשר בין המחשב ל-GPU היה צורך להתקין מספר דרייברים מהאתר של Nvidia:

- **CUDA Toolkit (Compute Unified Device Architecture)** - זוהי ארכיטקטורת מחשב מקבילי שפותחה ע"י חברת Nvidia, אשר מפתחת מעבדים גרפיים. CUDA מספקת את הכלים והמסגרת הבסיסיים לתכנות וביצוע חישובים מקבילים ב-GPU's. NVIDIA המאפשר למפתחים לגשת לזיכרון ולערכת הפקודות של המכשיר דרך שפת C for CUDA, מה שהופך אותו לאידיאלי עבור משימות MPP (Massively Parallel Programming). עבור הפרויקט שלנו התקנו את גרסת CUDA 11.8.
- **cuDNN (CUDA Deep Neural Network Library)** - על-מנת שנוכל להשתמש בספריית TensorFlow המבוססת על GPU נדרשת התוכנה שתגשר בין הספרייה ל-GPU. ספרייה זו חיונית למיטוב ביצועי למידה עמוקה ע"י אספקת יישומים מכוונים במיוחד כגון: קונבולוציה, נורמליזציה, pooling והפעלת שכבות. עבור הפרויקט שלנו התקנו את גרסת cudnn-11.2-windows-x64-v8.1.0.77.
- **TensorFlow** - לאחר התקנת CUDA ו-cuDNN התקנו את TensorFlow. זוהי ספריית קוד פתוח שפותחה ע"י גוגל, לבנייה ואימון של רשתות נוירונים. TensorFlow נמצא בשימוש נרחב הן למחקר והן לפיתוח וכולל API(Keras) לתכנות בשפות כמו C ו-Python. ספריית TensorFlow הותקנה באמצעות 'pip', מנהל חבילות המקל על ההתקנה והניהול של ספריות python כמו Numpy וכו'. החישובים בספרייה זו מיוצגים ע"י זרימת מידע

דרך גרף מצבים. המידע שזורם בגרף מיוצג ביחידות של טנזורים (מערכים רב-ממדיים). מכאן גם נובע השם שלה כהלחמה של המילה 'Tensor', מערך רב-ממדי ו-'Flow', זרימה. נשים לב שספריית ה-TensorFlow עם תמיכה ב-GPU נתמכת ב-Windows עד Python 3.9 עקב אי-הסכמה בין מפתחי הספרייה ל-Microsoft, כמו כן גרסת ספריית ה-TensorFlow היא 2.16.1.

- הערה- סביבת העבודה שלנו (IDE), הינה PyCharm.

כדי לבדוק שה-GPU קושר כראוי, הרצנו את הרשת וראינו ב"מנהל משימות" שאכן ה-GPU עובד כפי שנראה באיור 9.



איור 9 בדיקת תקינות ה-GPU

Dataset MNIST 4.2

כחלק מהצורך בבדיקה האם הרשת עובדת כראוי, בחרנו להשתמש ב-Dataset של MNIST. מסד הנתונים של MNIST (Modified National Institute of Standards and Technology database), הינו מסד נתונים גדול של ספרות בכתב-יד המשמש בדרך-כלל להכשרת מערכות שונות לעיבוד תמונה. מערך האימון מכיל 60,000 תמונות ו-10,000 תמונות בדיקה. כל תמונה היא תמונה בגווני אפור בגודל 28x28 פיקסלים, כאשר כל ערך פיקסל נע בין 0 (שחור) ל-255 (לבן).

מערך הנתונים של MNIST משמש לעיתים-קרובות כמדד להערכת ביצועים של מודלים שונים של למידת מכונה, במיוחד במשימות הכוללות סיווג תמונות, זיהוי תבניות ולמידה עמוקה.

בהקשר של רשתות Encoder-Decoder, ניתן להשתמש במערך הנתונים של MNIST כדי לאמן מודל לקודד (encode) ולאחר-מכן לשחזר (decode), תמונות של ספרות בכתב-יד. שימוש ב-MNIST ברשת שלנו, מאפשר לנו לאמת את יכולת הרשת לטפל בנתוני תמונה, וליישם טכניקות דומות על מערכי נתונים מורכבים יותר ויישומיים בעולם האמיתי.

4.3 הדמיית טשטוש (Blur)

בחלק זה של הפרויקט, לקחנו תמונות ממערך הנתונים MNIST, כדי לאמת את הפונקציונליות של הרשת שלנו. ע"י הצגת טשטוש המדמה את ההשפעה של רקמה מפזרת, אנו יכולים להעריך באיזו יעילות הרשת יכולה לשחזר תמונות.

את הצגת הטשטוש יישמנו ע"י קטע הקוד הבא (איור 10):

```
Method that take image and blur it with GaussianBlur with different sigma_x values
Then it will blur the image with 11X11 matrix.
Input: regular image without filter
Output: Blur Image
"""
def gaussian_m (image):

    blurred_img=image
    for i in range(25):
        # Increase kernel size and sigma for stronger blurring
        #kernel_size = (5 + i*2, 5 + i*2)
        sigma_x = 10+2*i
        blurred_img = cv2.GaussianBlur(blurred_img, (15,15), sigma_x, cv2.BORDER_TRANSPARENT)
        blurred_img=cv2.blur(blurred_img,(11,11))
    return blurred_img
```

איור 10 קטע קוד המבצע טשטוש

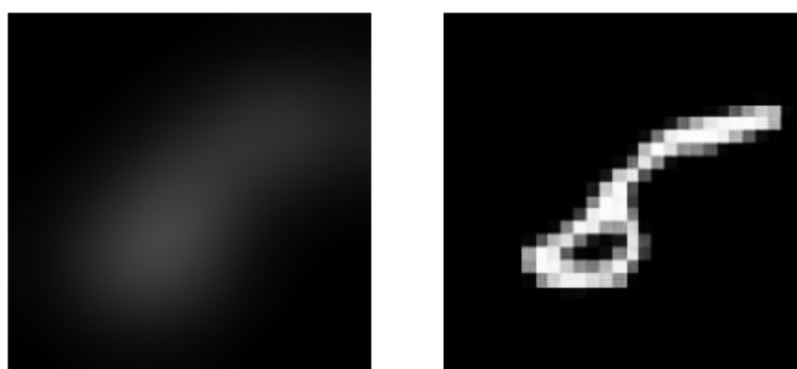
בקוד זה השתמשנו בפונקציה gaussian_m. פונקציה זו לוקחת תמונה נתונה ומבצעת עליה טשטוש גאוסיאני חוזר ונשנה. בכל איטרציה, הפונקציה מבצעת גם טשטוש ממוצע, כאשר לאחר 25 איטרציות נקבל את רמת הטשטוש הרצויה.

לאחר-מכן חילקנו את התמונות מה-Dataset לתיקיות של "תמונות מקוריות" ו"תמונות מטושטשות". באיור 11 ניתן לראות בצד שמאל את התמונות לפני הטשטוש (קרי- מקוריות), ובצד ימין ניתן לראות את התמונות לאחר הטשטוש.

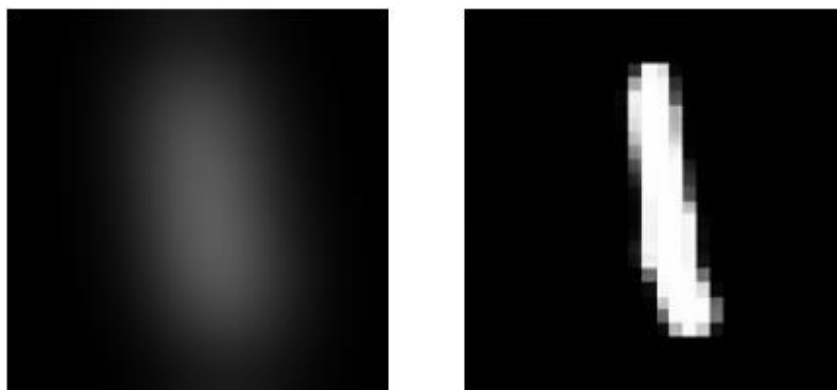


איור 11 משמאל: התמונות המקוריות, מימין: התמונות המטושטשות

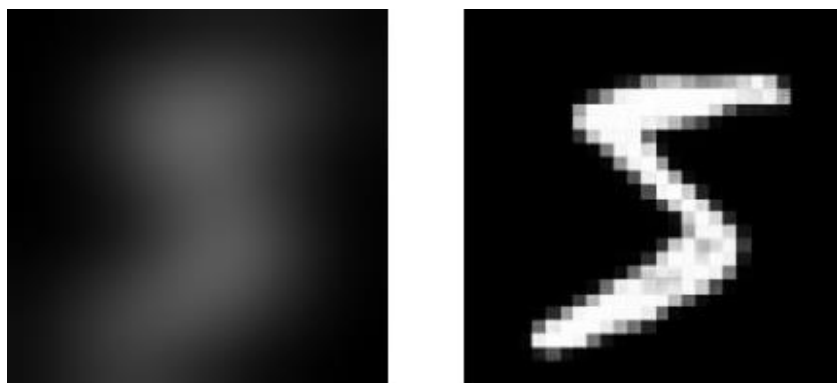
הערה: בחלק זה של הניסוי לא עשינו conv משחור ללבן לתמונות המקוריות. בשלב הבא, העברנו את כלל התמונות דרך הרשת של ה-Encoder-Decoder כדי לוודא שהרשת אכן מצליחה לפענח את התמונות המטושטשות. דוגמאות לתמונות שהרשת הצליחה לזהות (איורים 12, 13, 14, 15):



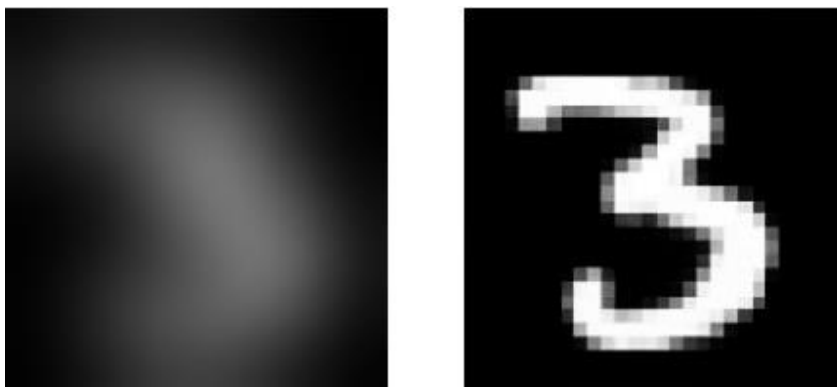
איור 12



איור 13



איור 14

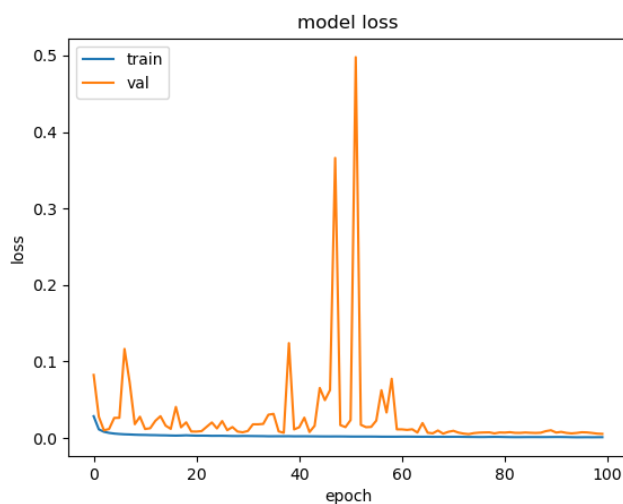


איור 15 באיור זה ובאיורים 12, 13 ו-14 ניתן לראות משמאל: את התמונות המטושטשות, ומימין: את פענוח הרשת

בתמונות ניתן לראות כי הרשת הצליחה לפענח את התמונות מתוך ה-dataset בצורה טובה ולהסיר את הטשטוש.

להלן יוצגו הגרפים של תוצאות הרשת (גרף 3, 4):

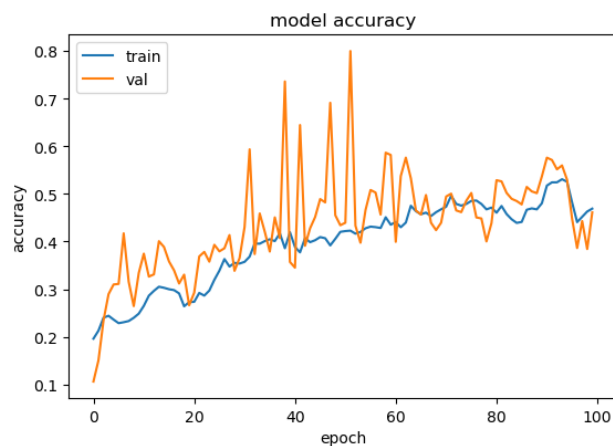
פונקציית Loss:



גרף 3 model loss

ניתן לראות בגרף 3 שפונקציית ה-Loss מתכנסת ב-Train וב-Validation, ולכן זהו אחד המדדים שהרשת עובדת כראוי.

Accuracy:



גרף 4 model accuracy

ניתן לראות בגרף 4 שמבחינת דיוק ה-Validation עוקב אחר ה-Train אך לא במידה רבה. ביישום המעשי של המערכת שינינו את הפרמטרים של הרשת כגון: learning rate ו-batch size, כדי לקבל תוצאות דיוק גבוהות יותר כפי שניתן יהיה לראות בגרף 6.

פרק 5: ניסוי שני – יישום מעשי של המערכת

הציוד לניסוי:

- RPI Module 4B
- CP2102 USB UART Board HW409
- Waveshare e-paper screen 4.3inch 800x600
- Basler Camera acA1440-220um
- נורת Led של חברת DURACEEL, 6500k, זווית תאורה 240° + בית נורה
- מפצל USB 3.0 של חברת Gold Touch
- מפצל שקעים
- קופסא שחורה לצורך דימוי תנאי מעבדה

לצורך הבנת אופן הפעולה, נסקור כל רכיב ומה שימושו:

RPI Module 4B 5.1

ה-RPI Module 4B הינו שדרוג משמעותי ביחס לגרסאות הקודמות, שנועד לספק ביצועים משופרים, קישוריות רב-תכליתיות עבור מגוון רחב של יישומים מאוטומציה ביתית ועד לפרויקטים של "עשה זאת בעצמך" ואפילו כאלטרנטיבה שולחנית בעלות נמוכה.



איור 16 Raspberry Pi Module 4B

מפרט טכני:

- מעבד-Broadcom BCM2711, Quad-core Cortex-A72 (ARM v8) 64-bit SoC
- מהירות שעון 1.8GHz
- 8GB LPDDR4-3200 SDRAM או 16GB ,4GB ,2GB
- 2.4GHz ו-IEEE 802.11ac 5GHz אלחוטי
- BLE ,Bluetooth 5.0
- Gigabit ethernet
- 2 יציאות USB 3.0, ו-2 יציאות USB 2.0
- 40 pin GPIO
- 2 יציאות micro-HDMI (נתמכות עד 4Kp60)
- יציאת תצוגה MIPI DSI דו-מסלולית
- יציאת תצוגה MIPI CSI דו-מסלולית
- יציאת שמע סטריאו ווידאו 4 קוטבים
- H.265 (פענוח 4Kp60), H.264 (פענוח 1080p60, קידוד 1080p30)
- OpenGL ES 3.1, Vulkan 1.0
- חריץ כרטיס SD לטעינת מערכת הפעלה ואחסון נתונים
- טמפרטורת עבודה- 0-50 מעלות צלזיוס

5.1.1 התקנת מערכת הפעלה ל-RPI

על-מנת שנוכל להפעיל את ה-RPI נדרשת התקנה של מערכת ההפעלה. התקנה של מערכת ההפעלה ל-RPI באמצעות Raspberry Pi Imager היא הדרך הקלה והמהירה כדי להתקין את מערכות ההפעלה השונות על כרטיס ה-SD. בפרויקט שלנו השתמשנו בכרטיס SD של 128GB כדי שיהיה לנו מספיק זיכרון לדגום 30,000 תמונות ולפענח אותם.



איור 17 Raspberry Pi Imager

שלבי ההתקנה:

- ראשית ניכנס לאתר של Raspberry Pi ונילחץ על הלשונית Software.
 - לאחר-מכן נבחר ב-Download for Windows (מערכת ההפעלה של המחשב שלנו).
 - לפני שנמשיך נוודא שכרטיס ה-SD מחובר למחשב.
 - ב-CHOOSE DEVICE נבחר ב-Raspberry Pi Model 4.
 - ב-CHOOSE OS נבחר ב-Raspberry Pi OS (64-bit).
 - ב-CHOOSE STORAGE נבחר בכרטיס ה-SD שחיברנו למחשב.
- לאחר שהורדנו את מערכת ההפעלה לכרטיס ה-SD נחבר אותו ל-RPI ונפעיל אותו. על הצג נקבל את מסך הבית של ה-RPI (איור 18):



איור 18 מסך הבית של RPI

לאחר האתחול הראשוני נצטרך להתקין עדכונים. נעשה זאת ע"י כתיבת הפקודה `Sudo apt-get update`. פקודה זו תתקין את כל העדכונים החשובים ל-RPI שלנו כדי שנוכל להתחיל להשתמש בו.

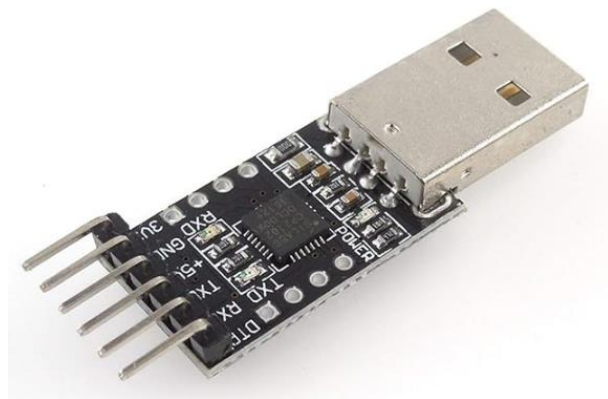
5.2 מודל תקשורת UART

מודל תקשורת UART (Universal Asynchronous Receiver-Transmitter), הוא פרוטוקול תקשורת חומרה המאפשר תקשורת טורית בין מכשירים. הוא נמצא בשימוש נרחב במערכות משובצות ופרויקטים מבוססי מיקרו-בקר בשל הפשטות והיעילות שלו.

תהליך התקשורת של UART פועל באופן הבא:

- המשדר ממיר נתונים מקבילים לזרם נתונים טורי.
- המשדר מוסיף את הסיביות הדרושות כדי ליצור את מסגרת הנתונים.
- המקלט מזהה את סיביות אלו כדי לאמת את הנתונים שהתקבלו.
- המקלט ממיר את זרם הנתונים הטורי בחזרה לנתונים מקבילים, בהם ניתן להשתמש במכשיר המקבל.

תקשורת UART היא תקשורת א-סינכרונית, כלומר המשדר והמקלט אינם חולקים אות שעון משותף. הם מסתמכים על תצורת קצב ההעברה כדי להבטיח סנכרון תקין במהלך שידור וקליטה של נתונים. בפרויקט שלנו השתמשנו ב-CP2102 USB UART HW409, המשמש כגשר בין יציאת ה-USB של ה-RPI למסך ה-Waveshare ומאפשר תקשורת טורית חלקה.



איור 19 CP2102 USB UART HW409

לרגלי ה-CP2102 USB UART ישנם 6 רגליים:

1. *DTR* - צורה אחרת של בקרת זרימת החומרה.
2. *RXD* - קלט נתונים אסינכרוני
3. *TXD* - פלט נתונים אסינכרוני

4. $+5V$ - פין יציאה VCC
5. GND - פין זה מספק את ההארקה המשותפת לשני המכשירים.
6. $3V3$ - פין יציאה VCC

5.2.1 התקנת הדרייבר ל-UART

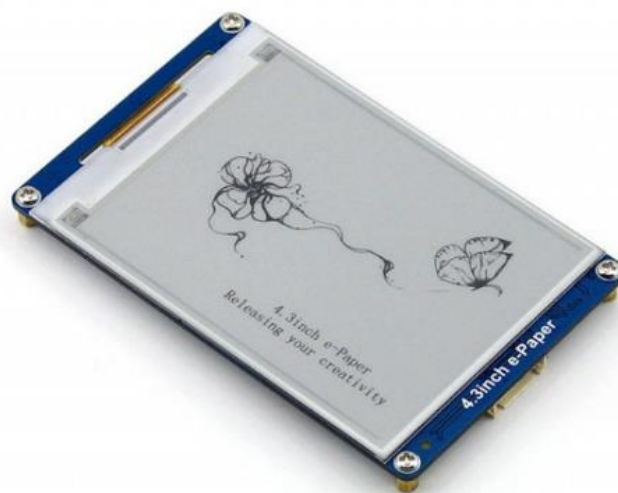
כדי לאפשר למערכת ההפעלה לזהות ולתקשר עם ממשק ה-UART CP2102 בצורה יעילה, וכן להשתמש במכשירים חיצוניים המחוברים, נדרש להתקין את הדרייבר המתאים (מנהל התקן).

הוראות התקנה:

- יש להיכנס לאתר של SILICON LABS תחת USB to UART Bridge VCP Drivers
- ב-Download יש לבחור ב-CP210x Windows Drivers
- נפתח את הקובץ ונבחר ב-CP210xVCPInstaller_x64
- ניכנס ל"מנהל ההתקנים" ונראה תחת יציאת COM את ה-CP2102 Module

5.3 מסך ה-Waveshare

מסך ה-Waveshare זהו מודול תצוגת E-Ink ממשק טורי, בעל מסך בגודל 4.3Inch ורזולוציה של 800x600.



איור 20 Waveshare e-paper screen 4.3inch 800x600

מפרט טכני:

- למסך יש זיכרון NandFlash מובנה בעל נפח 128MB, כמו-כן ניתן לאחסן את התמונות שישמשו אותנו בניסוי גם בכרטיס חיצוני (SD card).
- המסך יכול להציג תמונות ברמות אפור של 1bpp ו-2bpp.
- קצב ההעברה בממשק טורי הוא: 115200bps שמוגדר כברירת מחדל בעת הפעלת המסך.
- המסך יכול לעבוד ברמות מתח של בין 3.3v ל-5v.
- כשהמסך במצב שינה הזרם יהיה פחות מ-5mA.
- ניתן לשלוט במסך ע"י תוכנה פנימית של Waveshare הנקראת EPSCOMM.

למסך ישנם 6 יציאות:

1. VCC: 3.3V ~ 5V
2. GND- הארקה
3. DOUT- העברת נתונים ביציאה טורית
4. DIN- קליטת נתונים ביציאה טורית
5. WAKE_UP-עירור חיצוני
6. RST- איפוס חיצוני

אופן פעולת המסך בעת הדלקה ראשונה:

לאחר חיבור המסך למקור מתח, המסך עובר תהליך של ריענון. ברגע זה, אם אין כרטיס SD חיצוני בשימוש, נורת המחווה תהבהב 3 פעמים. כאשר יש כרטיס SD חיצוני בשימוש, נורת המחווה תידלק מבלי שתהבהב. לאחר ההפעלה, המערכת קוראת את הגופן ואת נתוני התמונה מה-NandFlash הפנימי כברירת מחדל. בעת שימוש בכרטיס SD נצטרך למקם את התמונות בכרטיס החיצוני לפני השימוש בו.

אופן הצגת התמונות:

המערכת תומכת בתצוגת תמונות של 1bpp ו-2bpp, כמו-כן המערכת תומכת אך ורק בפורמט BMP. עבור פרמטרים אחרים של תמונה, נצטרך להמיר את התמונות הללו לפורמט BMP באחת משתי הדרכים הבאות:

- תוכנה ייעודית של חברת Waveshare (UC-GUI-BitmapConvert.exe)
 - ע"י קוד (בניסוי שלנו אנו נכתוב קוד בשפת התכנות Python שעליו יפורט בהמשך)
- בנוסף, ישנם כללים כיצד שם התמונה צריך להיות על-מנת שנוכל להציג את התמונה על המסך:
- שם התמונה חייב להיות באותיות **גדולות** ובאנגלית.
 - אורכו של השם יהיה פחות מ-10 תווים, כאשר "BMP". כלול בשם התמונה.

לדוגמה:

PIC7.BMP ו-PIC789.BMP, הינם שמות תקינים לתמונות.

PIC7890.BMP, הינו שם לא תקין לתמונה, מכיוון שמכיל 11 תווים.

חיבור המסך למודל תקשורת UART:

כאמור, השתמשנו במודל תקשורת מסוג CP2102 USB UART Board. הפינים שחיברנו בין המסך למודל תקשורת מוצגים בטבלה הבאה (טבלה 1):

CP2102 USB UART Board	Waveshare e-paper screen 4.3Inch 800x600
VCCIO	Red
GND	Black
RXD	White
TXD	Green

5.3.1 התקנת תוכנת EPSCOMM

ראשית, נצטרך מתוך עמוד הבית של חברת Waveshare להתקין את התוכנה שמפעילה ידנית את המסך. הורדת התוכנה תשמש אותנו לבדוק שאכן המסך יכול להעלות את התמונות כנדרש, ואת סדר הפעולות שנצטרך לעשות כאשר נפתח לו תקשורת ע"י ספריית Serial בשפת Python.

להלן ההוראות כיצד להוריד ולהפעיל את תוכנת EPSCOMM:

a. יש להיכנס לאתר של Waveshare ולהיכנס ללשונית 'Resources' ולבחור ב-

'Software'. לאחר-מכן להוריד את שלושת הקבצים ולקבץ אותם תחת אותה תיקייה:

1. 4.3Inch-e-paper-COM.72

2. UC-GUI-BitmapConvert.72

3. 4.3Inch-e-paper-TF

לאחר ההורדות של כל הקבצים התיקייה אמורה להראות כך (איור 21):

EN < EPSCOMM < 4.3inch-e-Paper-COM (2) < הורדות < (				
... תצוגה מיון				
שם	תאריך שינוי	סוג	גודל	
GBK32	22/02/2024 21:16	קובץ גופן	2,993 KB	
GBK48	22/02/2024 21:16	קובץ גופן	6,734 KB	
GBK64	22/02/2024 21:16	קובץ גופן	11,970 KB	
PIC2	22/02/2024 21:16	קובץ BMP	74 KB	
PIC3	22/02/2024 21:16	קובץ BMP	74 KB	
PIC4	22/02/2024 21:16	קובץ BMP	235 KB	
PIC7	22/02/2024 21:16	קובץ BMP	235 KB	
לפני זמן רב				
EPSCOMM_EN	28/03/2015 12:42	יישום	308 KB	
Royale.ssk	02/03/2006 06:33	קובץ SSK	75 KB	
SkinPPWTL.dll	09/12/2005 09:37	הרחבת יישום	537 KB	
register the control first	25/03/2005 00:05	Wind... קובץ אצווה של	1 KB	
uC-GUI-BitmapConvert	19/08/2004 03:08	יישום	43 KB	
MSCOMM32.OCX	23/06/1998 19:00	פקד ActiveX	102 KB	

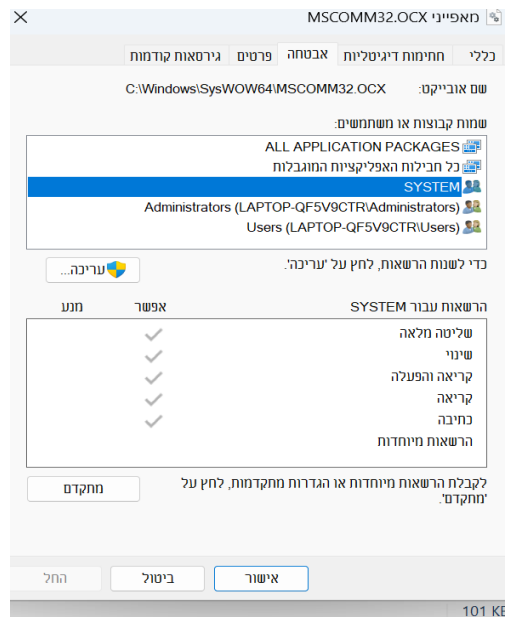
איור 21 הורדת שלושת הקבצים והכנסה לתיקייה אחת

b. בתוך התיקייה ישנו קובץ MSCOMM32.OCX. יש להעתיק אותו לתוך התיקיות SysWOW64 ו-Sys32 כמוצג באיור 22:

< Windows < Windows-SSD (C:) < מחשב זה < (				
... תצוגה מיון				
שם	תאריך שינוי	סוג	גודל	
SoftwareDistribution	27/02/2024 20:10	תיקיית קבצים		
Speech	07/05/2022 08:24	תיקיית קבצים		
Speech_OneCore	07/05/2022 08:24	תיקיית קבצים		
System	07/05/2022 08:24	תיקיית קבצים		
System32	27/02/2024 20:09	תיקיית קבצים		
SystemApps	28/07/2023 14:02	תיקיית קבצים		
SystemResources	14/02/2024 22:50	תיקיית קבצים		
SystemTemp	28/02/2024 16:20	תיקיית קבצים		
SysWOW64	25/02/2024 10:32	תיקיית קבצים		

איור 22 הוספת הקובץ MSCOMM32.OCX לתיקיות SysWOW64 ו-Sys32

c. בכל אחת מהתיקיות משלב 2, יש להגיע למאפיינים של הקובץ MSCOMM32.OCX, תחת "אבטחה" יש לוודא של-System יש את כל ההרשאות (מומלץ לעשות גם ל-users את כל ההרשאות במיוחד כאשר בשלב 5 ישנה שגיאה על המסך). לאחר מתן ההרשאות יש ללחוץ על "אישור" (איור 23).



איור 23 לאחר מתן ההרשאות יש ללחוץ על "אישור"

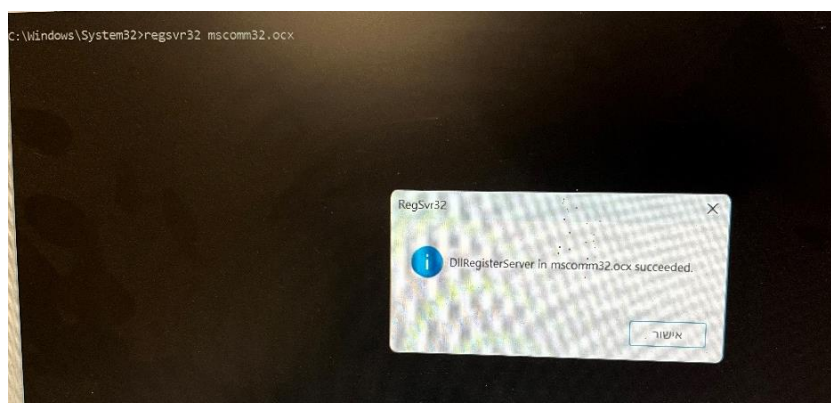
- d. אופציה- יש להוסיף את התוכנה לחומת האש של Windows, כדי שלא תיחסם.
 e. תחת התיקייה משלב 2, יש ללחוץ על 'register the control first', ניתן להפעיל כ"מנהל מערכת" (איור 24).

לפני זמן רב

308 KB	יישום	28/03/2015 12:42	EPSCOMM_EN
75 KB	קובץ SSK	02/03/2006 06:33	Royale.ssk
537 KB	הרחבת יישום	09/12/2005 09:37	SkinPPWTL.dll
1 KB	קובץ אצווה של Wind...	25/03/2005 00:05	register the control first
43 KB	יישום	19/08/2004 03:08	uC-GUI-BitmapConvert

איור 24 תחת התיקייה משלב 2, יש ללחוץ על 'register the control first'

על המסך נקבל (איור 25):



איור 25 חלונת המאשרת את ביצוע הפעולה

○ הערה חשובה: במידה ומופיע שגיאה, כנראה שצריך להריץ כ"מנהל מערכת",

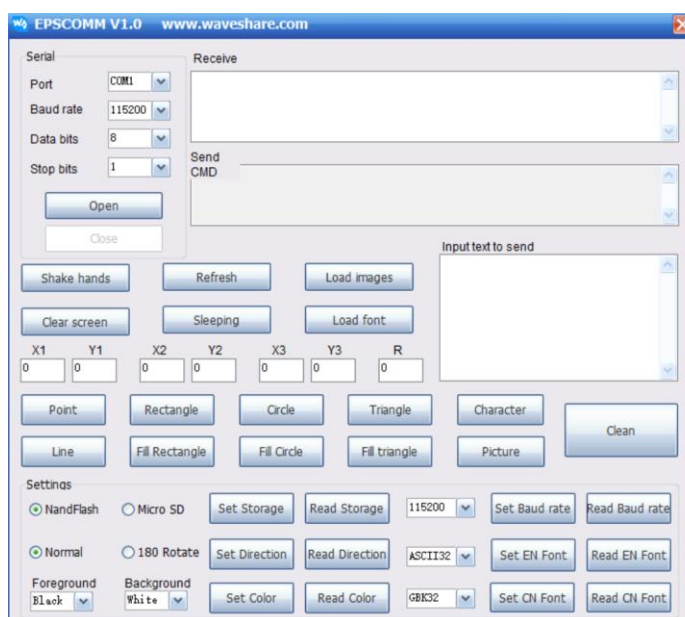
ללא קבלת האישור לא ניתן יהיה להריץ את התוכנה!!

f. נלחץ על התוכנה ונקבל (איור 26, 27):

לפני זמן רב

308 KB	יישום	28/03/2015 12:42	EPSCOMM_EN
75 KB	קובץ SSK	02/03/2006 06:33	Royale.ssk
537 KB	הרחבת יישום	09/12/2005 09:37	SkinPPWTL.dll
1 KB	Wind... קובץ אצווה של	25/03/2005 00:05	register the control first
43 KB	יישום	19/08/2004 03:08	uC-GUI-BitmapConvert

איור 26 תוכנת EPSCOMM



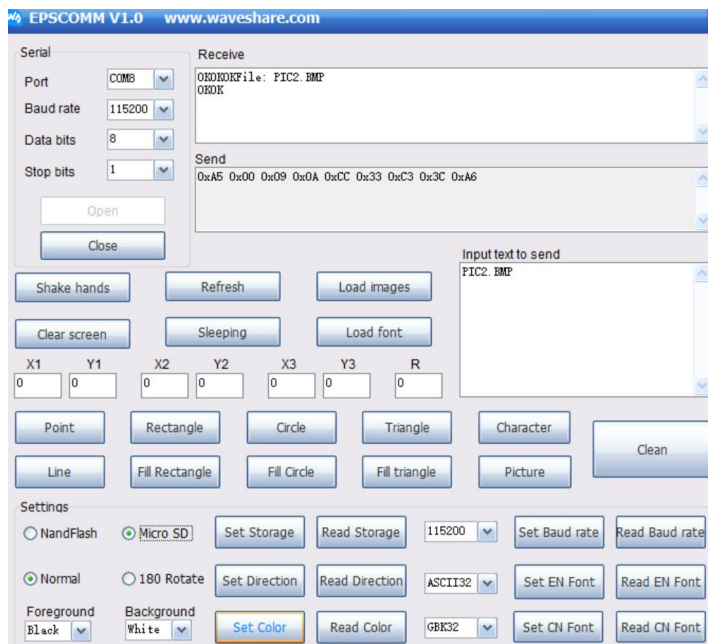
איור 27 חלונית תצוגת המסך

יש לשים לב כי הדרייבר של ה-UART מותקן על המחשב. במידה ולא מותקן הדרייבר על המחשב אין תקשורת בין המסך למחשב ולכן יופיע בצמוד לחיבור סימן שאלה כתום.

הפעלת התוכנה:

- בוחרים את port הכניסה
- לוחצים על Open
- Shake hands

- ב-setting לוחצים על microSD
- Set storage
- Load images
- כותבים את שם התמונה ב-'input text to send' עם סיומת BMP
- לוחצים על picture
- X_1 ו- Y_1 יאותחלו לאפס
- לוחצים על refresh



איור 28 ב-send ניתן לראות את שם התמונה ב-hexa

על המסך נקבל את התמונה שרצינו להציג (איור 29):



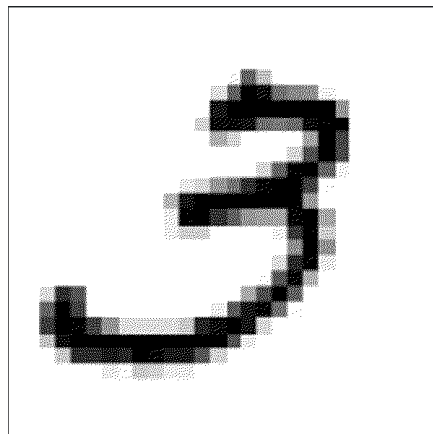
איור 29 תמונת בדיקה של חברת Waveshare

הערה: התמונה סופקה ע"י חברת WaveShare לצורך בדיקת העלאת תצוגה למסך.

5.3.2 הכנת מידע- מעבר לפורמט BMP

ראשית על-מנת שנוכל להציג את התמונות למסך, אנו נצטרך להעביר את התמונה לפורמט BMP כך שהעומק יהיה בגודל 1bpp. אנו נשתמש באותו dataset של MNIST, רק שהפעם נרצה שהאובייקט יהיה בצבע שחור והרקע יהיה בצבע לבן.

לפני שנבצע את ההמרה ל-1bpp, נצטרך להמיר את התמונה קודם ל-24bpp, מכיוון שללא המרה מקדימה ל-24bpp נקבל תמונה שאינה ברורה ושלא ניתנת להצגה על המסך, כפי שניתן לראות באיור 30.



איור 30 ביצוע המרה ל-1bpp ללא המרה מקדימה ל-24bpp

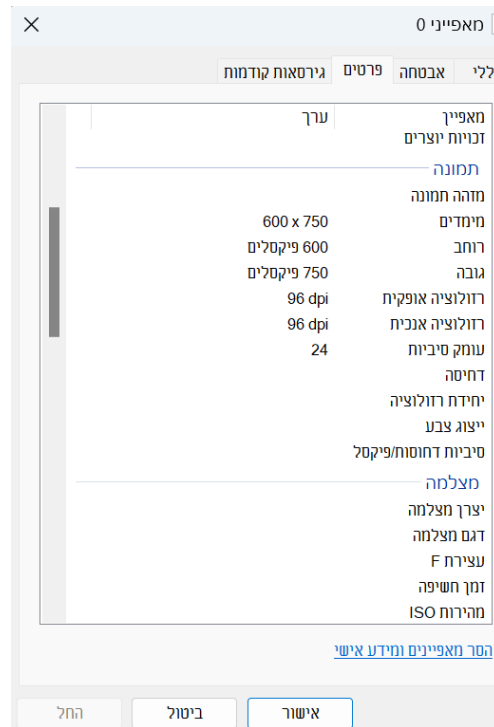
שלבים להמרת תמונה ל-1bpp:

בשלב הראשון הורדנו את ה-dataset של MNIST ועשינו לו resize של 600x750, כפי שניתן לראות באיור 31.

```
35 ▶ if __name__ == "__main__":
36     # Load MNIST dataset
37     mnist = tf.keras.datasets.mnist
38     (train_images, train_labels), (test_images, test_labels) = mnist.load_data()
39
40     # Resize and save the first 2 training images as JPG in the desired format
41     save_dir = os.path.join(os.getcwd(), 'mnist_converted')
42     re_size_img = (600, 750)
43
44     # Save the first 30000 images for demonstration
45     save_mnist_images_as_jpg(train_images[:30000], save_dir, re_size_img)
```

איור 31 הורדת ה-dataset של MNIST וביצוע resize ל-600x750

בשלב השני פתחנו את התמונה ע"י PIL והמרנו את התמונה ל-Byte (8-bit). לאחר-מכן המרנו את התמונה כך שהמספר יהיה בצבע שחור והרקע יהיה בצבע לבן ושמרנו את התמונה כ-24bpp, כפי שניתן לראות באיור 32.



איור 32 המרת התמונה ל-24bpp

בשלב השלישי המרנו את התמונה ל-1bpp כדי שנוכל להציג אותה על מסך ה-Waveshare. השיטה תקבל כקלט:

- נתיב להיכן שהתמונות נמצאות בפורמט 24bpp.
- רשימה של התמונות בפורמט 24bpp.
- נתיב לשמירה של התמונות לאחר ההמרה משחור ללבן.

פלט השיטה יהיה:

- שינוי רזולוציה ל-600x600.
- המרה ל-1bpp.
- שינוי מפורמט jpg ל-BMP.

ניתן לראות באיור 33 כיצד יישמנו את ההמרה בקוד :

```
14 def change_name(path, photo_files, bmp_path):
15     # Create bmp_path directory if it doesn't exist
16     os.makedirs(bmp_path, exist_ok=True)
17
18     for i, photo_file in enumerate(photo_files):
19         old_name = os.path.basename(photo_file)
20         old_path = os.path.join(path, old_name)
21         img = Image.open(old_path)
22         img = img.resize((600, 600)) #Resized to 600x600
23         img_bw = img.convert(mode='1') # Images converted to 1bpp
24
25         # Save the image as BMP format
26         new_img = f"I{i}"
27         new_path = os.path.join(bmp_path, new_img)
28
29         img_bw.save(new_path + ".bmp") #Saved in .bmp format
30         print(f"Converted '{old_name}' to '{new_img}.bmp'.")
31
```

איור 33 המרת התמונה ל-1bpp

5.3.3 הצגת התמונות על המסך

בשלב הראשון נפתח תקשורת טורית ע"י ספריית serial של Python, כדי שנוכל לשלוט על המסך מהמחשב (איור 34).

```
# Open the serial connection
print("Opening serial connection...")
ser = serial.Serial('/dev/ttyUSB0', 115200)
print("Serial connection opened.")
```

איור 34 פתיחת תקשורת טורית

לאחר-מכן, מתוך אתר היצרן ישנם מספר פקודות קבועות ב-hexa שהם מפעילות את המסך באופן תדיר, ובלעדיהן המסך לא יעבוד כשורה (איור 35):

```
shake_hands = "A5 00 09 00 CC 33 C3 3C AC"
set_storage = "A5 00 0A 07 01 CC 33 C3 3C A9"
read_storage = "A5 00 09 06 CC 33 C3 3C A6"
refresh = "A5 00 09 0A CC 33 C3 3C A6"
load_image = "A5 00 09 0F CC 33 C3 3C A3"
clear_screen = "A5 00 09 2E CC 33 C3 3C 82"
```

איור 35 הפקודות הקבועות של היצרן להפעלת המסך

- Shake hands - תחילת פתיחת תקשורת עם המסך (הזיהוי שלו).
- set_storage - מזהה את כרטיס הזיכרון שעליו התמונות שמורות.
- read_storage - קורא את התמונות מהזיכרון.
- Refresh - מבצע את הפעולה שהוזנה למסך.
- load_image - טוען את התמונות מכרטיס הזיכרון.
- clear_screen - פקודה לניקוי המסך מהתוכן המוצג עליו.

את כלל הפקודות ב-hexa אנו נשמור בקובץ שנקרא command_waveshare.txt, נהפוך כל פקודה ל-Bytes ונשדר אותה למסך.

כדי לזהות את הפקודה להעלאת תמונה למסך ב-hexa ישנם מספר שלבים שאותם נבצע:

- המרת אורך ה-frame ל-hexa.
- המרת שם התמונה ל-hexa (לפי טבלת ASCII).
- המרת ה-Parity byte ל-hexa.

בטבלה הבאה (טבלה 2) ניתן לראות את הפורמט של הפקודה:

	Standing command				Standing command	
Description	Frame header	Frame length	Command type	Parameter or data	Frame end	Parity Byte
Command	0xA5	0xXX XX	0xXX	0xXX....	0xCC 33 C3 3C	0xXX
Byte length	1Byte	2Byte	1Byte	0~1024Byte	4Byte	1Byte

כדי להציג כל תמונה בפורמט המתאים לה, נצטרך ליצור מספר שיטות שיקלו על התהליך (איור 36):

```

5 # convert each name to hexa
6 # example: IM0-> "49 4D 30"
1 usage
7 def convert_ascii_hexa(name):
8     result=""
9     for ch in name:
10         char_convert_hexa= format(ord(ch), "X")
11
12         result+=char_convert_hexa
13     spaced_string = ' '.join(result[i:i + 2] for i in range(0, len(result), 2))
14     return spaced_string
15
1 usage

```

איור 36 המרת תמונה מ-ASCII ל-hexa

שיטה זו תמיר את שם התמונה ל-hexa ע"פ טבלת ASCII כמו שניתן לראות באיור 37 :

dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char
0	0	000	NULL	32	20	040	space	64	40	100	@	96	60	140	`
1	1	001	SOH	33	21	041	!	65	41	101	A	97	61	141	a
2	2	002	STX	34	22	042	"	66	42	102	B	98	62	142	b
3	3	003	ETX	35	23	043	#	67	43	103	C	99	63	143	c
4	4	004	EOT	36	24	044	\$	68	44	104	D	100	64	144	d
5	5	005	ENQ	37	25	045	%	69	45	105	E	101	65	145	e
6	6	006	ACK	38	26	046	&	70	46	106	F	102	66	146	f
7	7	007	BEL	39	27	047	'	71	47	107	G	103	67	147	g
8	8	010	BS	40	28	050	(	72	48	110	H	104	68	150	h
9	9	011	TAB	41	29	051	)	73	49	111	I	105	69	151	i
10	a	012	LF	42	2a	052	*	74	4a	112	J	106	6a	152	j
11	b	013	VT	43	2b	053	+	75	4b	113	K	107	6b	153	k
12	c	014	FF	44	2c	054	,	76	4c	114	L	108	6c	154	l
13	d	015	CR	45	2d	055	-	77	4d	115	M	109	6d	155	m
14	e	016	SO	46	2e	056	.	78	4e	116	N	110	6e	156	n
15	f	017	SI	47	2f	057	/	79	4f	117	O	111	6f	157	o
16	10	020	DLE	48	30	060	0	80	50	120	P	112	70	160	p
17	11	021	DC1	49	31	061	1	81	51	121	Q	113	71	161	q
18	12	022	DC2	50	32	062	2	82	52	122	R	114	72	162	r
19	13	023	DC3	51	33	063	3	83	53	123	S	115	73	163	s
20	14	024	DC4	52	34	064	4	84	54	124	T	116	74	164	t
21	15	025	NAK	53	35	065	5	85	55	125	U	117	75	165	u
22	16	026	SYN	54	36	066	6	86	56	126	V	118	76	166	v
23	17	027	ETB	55	37	067	7	87	57	127	W	119	77	167	w
24	18	030	CAN	56	38	070	8	88	58	130	X	120	78	170	x
25	19	031	EM	57	39	071	9	89	59	131	Y	121	79	171	y
26	1a	032	SUB	58	3a	072	:	90	5a	132	Z	122	7a	172	z
27	1b	033	ESC	59	3b	073	;	91	5b	133	[	123	7b	173	{
28	1c	034	FS	60	3c	074	<	92	5c	134	\	124	7c	174	
29	1d	035	GS	61	3d	075	=	93	5d	135	]	125	7d	175	}
30	1e	036	RS	62	3e	076	>	94	5e	136	^	126	7e	176	~
31	1f	037	US	63	3f	077	?	95	5f	137	_	127	7f	177	DEL

www.alpharithmetic.com

איור 37 טבלת ASCII

למשל IM0 יהפוך ל-49 4D 30x0.

לאחר-מכן נשתמש בשיטה שתספור את אורך הפקודה ע"פ הרווחים (איור 38) :

```

16 # Methods will count the length of the command by spaces
17 usage
18 def count_len_command(image_command):
19     count=0
20     for i in image_command:
21         if (i==' '):
22             count+=1
23     count+=1
24     count_in_hex = format(count, 'X')
25     return count_in_hex

```

איור 38 חישוב אורך הפקודה

1. ראשית ניקח את שם התמונה ללא פורמט BMP ונמיר אותה ל-hexa.
2. לאחר-מכן אנו נגיע ל-Frame length המורכב ממספרי hexa קבועים ושם תמונה (איור 39).

```

name_photo = os.path.basename(img)
name_photo = os.path.splitext(name_photo)[0] # take the first part in IM0.BMP --> take: IM0
name_photo = me.convert_ascii_hexa(name_photo)
length_command = me.count_len_command(f"A5 00 00 70 00 00 00 00 {name_photo} 2E 42 4D 50 00 CC 33 C3 3C 00")

```

איור 39 קוד המראה את מספרי ה-hexa הקבועים ושם תמונה משתנה

3. לאחר-מכן כדי לחשב את ה-Parity Byte השיטה תעשה XOR בין כל חלקי הפקודה (איור 40, 41):

```

# Methods that calculate the parity byte by xor
1 usage
2 def cal_parity_byte(image_command):
3     first_command = 0xA5 # Assuming first command is represented as an integer
4     res_xor = first_command
5
6     # Convert the image command to a list of integers representing bytes
7     command_without_space = [int(byte, 16) for byte in image_command.split()]
8
9     # Perform XOR operation on each byte
10    for byte in command_without_space[1:-1]:
11        res_xor ^= byte
12    res_xor_in_hex = format(res_xor, 'X')
13    return res_xor_in_hex

```

איור 40 חישוב ה-Parity Byte

```

42 def xor_strings(str1, str2):
43     # if not same len xor will not works
44     if len(str1) != len(str2):
45         raise ValueError("Strings must have the same length for XOR operation")
46
47     # Convert each character to its Unicode value, perform XOR, and convert back to character
48     result = ''.join(chr(ord(char1) ^ ord(char2)) for char1, char2 in zip(str1, str2))
49     return result

```

איור 41 ביצוע XOR

4. לבסוף נשלח את הפקודה מקובץ הטקסט למסך ע"י המרה מ-hexa ל-Byte (איור 42).

```

51 # Function to send commands from a file
52 3 usages
53 def send_commands_from_file(file_path, ser):
54     with open(file_path, 'r') as file:
55         lines = file.readlines()
56         for line in lines:
57             command = bytes.fromhex(line.strip()) # Convert the hex string to bytes
58             ser.write(command) # Send the command

```

איור 42 שליחת הפקודה מקובץ הטקסט למסך

5. בשלב הבא, נתחיל ליצור לכל תמונה את הפקודה שלה ב-hexa.

המבנה הכללי של הפקודה ב-hexa שיישלח לקובץ command_waveshare.txt יראה כך :

A5 00 Length_of_command 70 00 00 00 00 Name_of_image 2E 42 4D 50 00 CC C3 3C
Parity_Byte

כדי להקל על החישובים נחשב את ה-Parity Byte בסוף ובינתיים נשים אותו כ-'00' ואת אורך הפקודה גם כ-'00'. אורך הפקודה יחושב ע"פ כלל המבנה של הפקודה עצמה כך :

$$(5.1) \quad \text{num of words} = \text{num of space} + 1$$

למשל:

נניח שהפקודה שלנו היא AA BB CC, נשים לב כי מספר הרווחים הינו 2 ומספר המילים הם 3 :
 $2+1=3$

באיור 43 ניתן לראות את שלבי המעבר ל-hexa כאשר המשתנים בשורת הפקודה הם : אורך הפקודה, שם התמונה וה-Parity Byte.

```

87     for img in photo_files:
88
89         start_time = time.time()
90         name_photo = os.path.basename(img)
91         name_photo = os.path.splitext(name_photo)[0] # take the first part in IM0.BMP --> take: IM0
92         name_photo = me.convert_ascii_hexa(name_photo)
93         length_command = me.count_len_command(f"A5 00 00 70 00 00 00 00 {name_photo} 2E 42 4D 50 00 CC 33 C3 3C 00")
94         parity_byte = me.cal_parity_byte(f"A5 00 {length_command} 70 00 00 00 00 {name_photo} 2E 42 4D 50 00 CC 33 C3 3C 00")
95         image = f"A5 00 {length_command} 70 00 00 00 00 {name_photo} 2E 42 4D 50 00 CC 33 C3 3C {parity_byte}"

```

איור 43 שלבי המעבר ל-hexa

6. לבסוף ניצור רשימה עם הפקודות של clear_screen, image ו-refresh ונכתוב אותם בקובץ הטקסט.

```

95     image = f"A5 00 {length_command} 70 00 00 00 00 {name_photo} 2E 42 4D 50 00 CC 33 C3 3C {parity_byte}"
96
97     # Commands for each photo
98     command_for_each_photo = [clear_screen, image, refresh]
99     with open(file_of_command_path, 'w') as f: # Use 'w' to overwrite the file
100         for command in command_for_each_photo:
101             f.write(command + '\n')
102
103     # Send all commands from the file for each photo
104     me.send_commands_from_file(file_of_command_path,ser)

```

איור 44 רשימת הפקודות של clear screen, image ו-refresh

איך הקוד עובד?

- ראשית, כמו שהוצג לעיל נפתח תקשורת טורית עם המסך ולאחר-מכן ניצור אובייקט למצלמה.
- נכוון את ה-Exposure Time להיות אוטומטי-רציף כדי שתוכל להתרגל לשינויי תאורה.
- נעתיק את כלל התמונות מה-Raspberry Pi לתוך רשימה.
- נעבור תמונה-תמונה ועל כל תמונה ניצור פקודה משלה.
- נשלח בסדר הבא לקובץ command_waveshare.txt את הפקודות הבאות:
 - Clear screen - לניקוי המסך מהתמונה הקודמת.
 - Image - הפקודה של התמונה שתוצג על המסך.
 - Refresh - כדי להציג את התמונה נעשה ריענון למסך כדי שהתמונה תוצג.

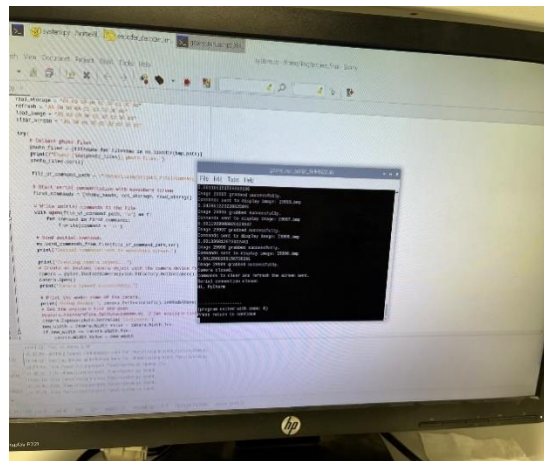
הערה: במידה ולא ננקח את המסך, ייווצר מצב שהתמונה החדשה תוצג על התמונה הקודמת ולכן יש חשיבות רבה לנקות את המסך בין תמונה לתמונה.

5.3.4 סגירת משאבים

לאחר הדגימה של כל התמונות, אנו נדאג לסגור את כלל המשאבים של המערכת, כלומר, תתבצע סגירה של התקשורת הטורית למסך, סגירת הקובץ וסגירה של המצלמה.

סגירת משאבים בקוד היא פעולה חשובה מכמה סיבות עיקריות :

- **ניהול זיכרון** - כאשר פותחים קובץ, חיבור לרשת, או משאב אחר, הוא תופס זיכרון ומשאבי מערכת אחרים. סגירה של משאבים אלו מבטיחה שהזיכרון הזה משתחרר, ובכך מונעת דליפות זיכרון שעלולות לגרום לבעיות בביצועים ובתפקוד של התוכנית.
- **מניעת התנגשויות** - אם משאב נשאר פתוח, ייתכן שמשתמשים אחרים או תהליכים אחרים לא יוכלו לגשת אליו. לדוגמה, קובץ שפתוח על-ידי תוכנית אחת לא יכול להיפתח לעריכה על ידי תוכנית אחרת, עד שהוא נסגר.
- **שיפור ביצועים** - סגירת משאבים עוזרת לשמור על ביצועים טובים של התוכנית. שמירה על כמות גדולה של משאבים פתוחים עלולה להאט את ביצועי המערכת.
- **שמירה על נכונות הנתונים** - סגירת הקובץ תמנע איבוד מידע.
- **שמירה על אבטחה** - משאבים פתוחים עלולים להיות פגיעים למתקפות אם אינם נסגרים כראוי.



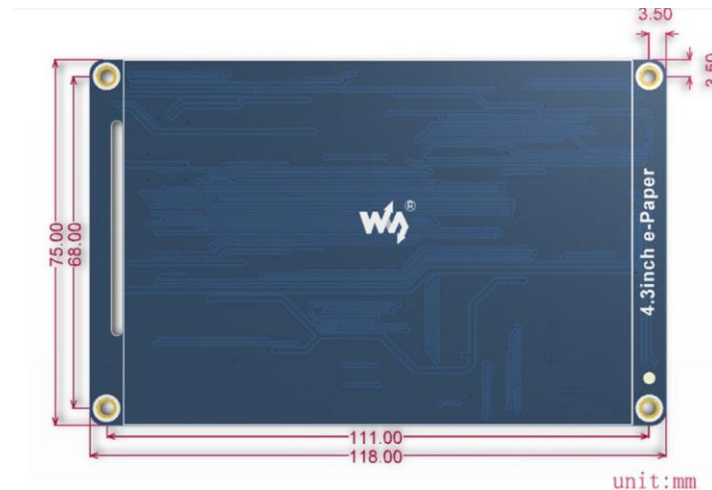
איור 45 סגירת המשאבים

5.3.5 קיבוע למסך ה-Waveshare

על-מנת שדגימת התמונות תהיינה בצורה מיטבית ובלי הפרעות של רעשים, נדרש שמסך ה-Waveshare יעמוד בצורה יציבה על המשטח האופטי. כדי להעמיד אותו בצורה יציבה, השתמשו בתוכנת FreeCAD בגרסה 0.21 לתכנון רגליים מתאימות למסך.

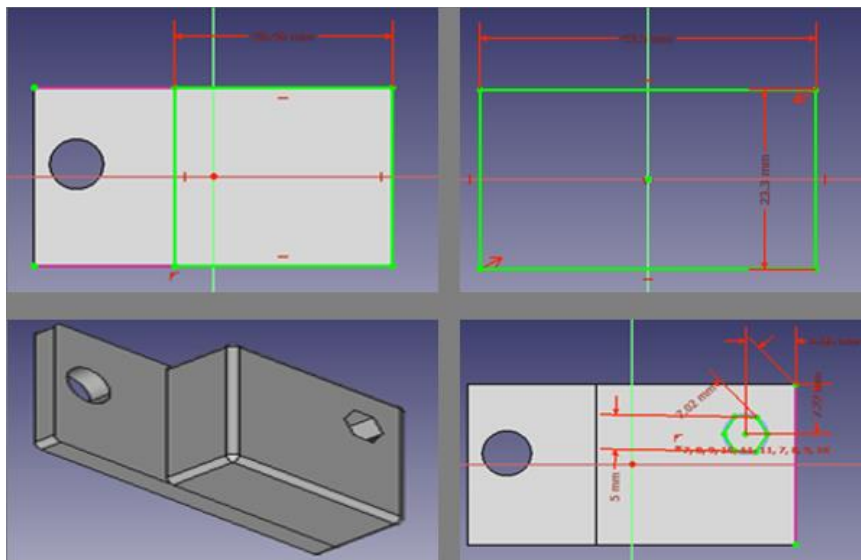
תוכנת FreeCAD זוהי תוכנת קוד פתוח המשמשת לעיצוב אובייקטים תלת-ממדיים בכל גודל. תוכנה זו פופולרית במיוחד בקרב מהנדסים, מעצבים וחובבי הדפסת תלת-מימד עבור הנדסת מכונות, עיצוב מוצר ויישומים טכניים אחרים.

את מידות המסך לקחנו מתוך אתר היצרן של חברת Waveshare (איור 46) :



איור 46 מידות מסך ה-Waveshare

כמו-כן לקחנו מידות של המשטח האופטי ובהתאם לכך תיכננו את הרגליים של המסך, כמו שניתן לראות באיור 47:



איור 47 תכנון רגל ימנית עליונה למסך

5.4 מצלמת Basler

מצלמת Basler הינה מצלמה תעשייתית בעלת ביצועים גבוהים הנמצאת בשימוש נרחב ביישומים הדורשים דיוק ואמינות כגון: ראייה ממוחשבת, הדמיה רפואית ובדיקת איכות. מצלמות אלו ידועות בעמידותן בתנאי סביבה קשים, ברזולוציה גבוהה ובקצבי פריימים גבוהים, אשר הופכים אותן למתאימות לשימוש בסביבות תובעניות כמו מפעלים ומעבדות מחקר.

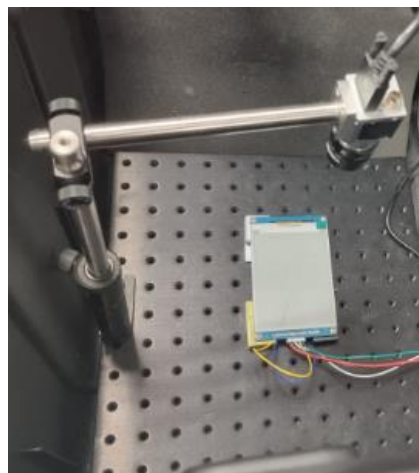


איור 48 Basler Camera acA1440-220um

מפרט טכני:

- רזולוציה - 1.6MP
- סוג חיישן - Sony IMX273LLR-C
- פורמט חיישן - 1/2.9"
- קצב פריימים - 227fps
- ממשק נתוני תמונה - USB 3.0, תומך ב-GigE (Gigabit Ethernet)
- Exposure Time - ניתן לתכנות באמצעות ממשק API של המצלמה
- הספק מצלמה - 5VDC, $\approx 3.3W$
- טמפרטורת עבודה - 0-50°C
- טמפרטורת Housing מקסימלית - 70°C
- Software - Basler pylon גרסה 4.0 ומעלה
- בפרויקט שלנו השתמשנו בעדשה בעלת אורך מוקד של 8mm

כדי שנוכל לדגום את התמונות מהמסך השתמשנו במצלמת Basler מדגם acA1440-220um שאותה מיקמנו על המשטח האופטי בזווית של 90° למסך ה-Waveshare, כפי שניתן לראות באיור 49:



איור 49 מיקום המצלמה

הערה: כדי לאפשר אינטגרציה של ה-Raspberry Pi עם המסך והמצלמה השתמשו במפצל USB 3.0 עם 2A (איור 50).

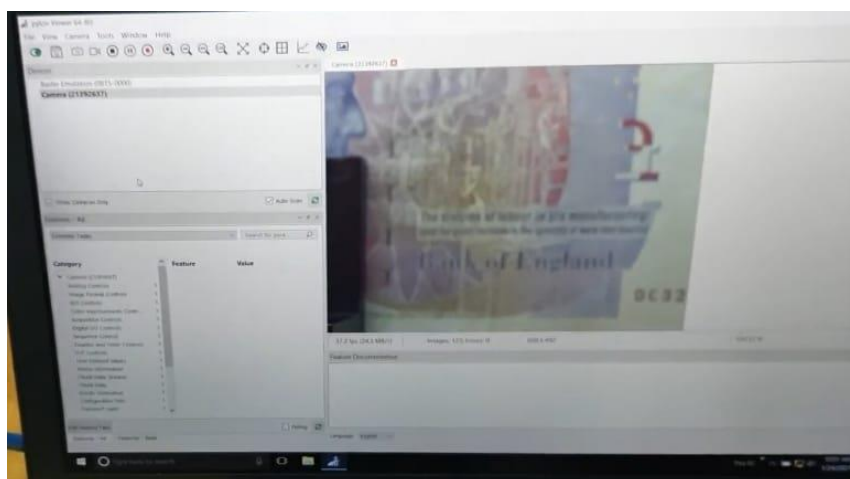


איור 50 מפצל USB 3.0

5.4.1 התקנת תוכנת pylon

Pylon זוהי חבילת תוכנה שמסופקת ע"י חברת Basler לשליטה ואינטראקציה עם המצלמות התעשייתיות שלהם. היא מאפשרת למפתחים לשלב את מצלמות Basler במגוון יישומים, מה שהופך אותה לכלי מפתח לגישה לפונקציות המצלמה, קביעת הגדרות ועיבוד תמונות.

עבור הפרויקט שלנו הורדנו מהאתר של חברת Basler את חבילת התוכנה PyPylon גרסה 4.0.0. לאחר התקנת התוכנה וחיבור של המצלמה למחשב, נוודא שהמחשב אכן מזהה את המצלמה כפי שניתן לראות באיור 51:



איור 51 תצוגת המסך של תוכנת Pylon

לאחר שהמחשב זיהה את המצלמה נעשה double-click על המצלמה, כדי שנוכל להשתמש במגוון הפונקציות של התוכנה. לחיצה על record תאפשר לנו צפייה ישירה במידע הוויזואלי המועבר מהמצלמה (איור 51), כך שנוכל להתאים ידנית את פרמטרי המצלמה להשגת תמונה ברורה ומוגדרת של האובייקט במסך ה-Waveshare.

5.5 תאורה

כאמור, את הניסוי ביצענו בתוך קופסא סגורה לצורך דימוי תנאי מעבדה. כדי שנוכל לדגום את התמונות ממסך ה-Waveshare, השתמשנו בנורת Led של חברת DURACELL בעלת טמפרטורת צבע של 6500k וזווית תאורה של 240° .

על-מנת שנוכל להשיג תמונות באיכות גבוהה, היה חשוב למצוא את הזווית והמרחק האופטימליים של מקור האור. מטרנתנו הייתה למנוע הן החזר אור עודף שעלול לגרום לתמונה לבנה, והן חוסר אור שיפגע בניגודיות ויגרום לתמונה חשוכה. לכן השתמשנו בבית נורה בעלת בסיס מתכוונן. פתרון זה אפשר לנו חופש תנועתיות כדי לשלוט בדיוק במיקום ובכיוון האור, למקד אותו באזור הרצוי ולמזער את ההשתקפויות כפי שניתן לראות באיור 52 :



איור 52 מימין: תאורת ה-Led, משמאל: תוצאת התמונה המתקבלת לאחר כיוון התאורה

5.6 מהלך הניסוי

לאחר השלמת הכנת כל המכשור הנדרש לניסוי, וביצוע בדיקות אימות לתפקוד תקין של ה-Raspberry Pi ומסך ה-Waveshare, ניתן להתקדם לשלב הביצוע של הניסוי.

5.6.1 רקמה מפזרת

בתחילת הניסוי הצגנו תמונה אקראית (לפני שידענו שצריך לעבוד עם ה-MNIST) במסך ה-Waveshare ללא טשטוש ומיקמנו את תאורת ה-Led במרחק מסוים ובזווית מסוימת, על-מנת לקבל מינימום של החזרות של אור לכיוון המצלמה כדי לקבל תמונה ברורה, כפי שניתן לראות באיור 53 :



איור 53 מבנה המערכת ללא טשטוש

לאחר-מכן הוספנו מדיום מפזר (במקרה שלנו טישו) למערכת הניסוי, שנועד לדמות פיזור של רקמה. בהתאם לכך, פיזור ניכר נגרם מקרני האור הנפלטות ממקור התאורה (נורת ה-Led). פיזור זה, המתאפיין בזווית פיזור רחבה, נובע מהאינטראקציה בין קרני האור לסיבים המרכיבים את המדיום המפזר. כתוצאה מכך, התקבלה התפלגות של עוצמת האור הנפלטת בזוויות שונות, בניגוד למצב בו לא נעשה שימוש במדיום מפזר, בו התקבלה התפלגות עוצמת אור אחידה יותר.

כדי להשיג תמונה ברורה בתנאי הפיזור החדשים, נדרשה אופטימיזציה גיאומטרית של מערכת התאורה, כלומר שינוי במיקום ובזווית של מקור האור ביחס למדיום המפזר ולמצלמה. שינויים אלו נועדו למזער את כמות האור המוחזר לכיוון המצלמה ולהבטיח תמונה ברורה יותר.

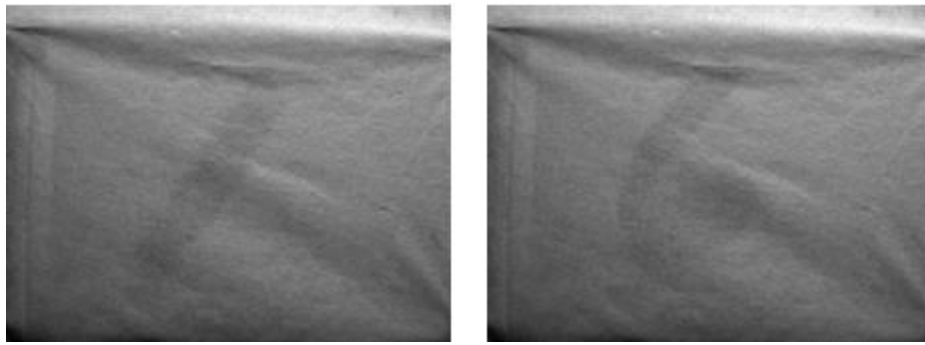
מכיוון שנרצה לקבל דיוק גבוה בחדות התמונה, נעזרנו בתוכנה של המצלמה, Pylon viewer, לקבלת המיקום האופטימלי של התאורה שימנע החזר של אור בצורה המקסימלית. את המהלך ביצענו על-ידי הרצה של הקוד המציג תמונות על מסך ה-Waveshare. באיור 54 ניתן לראות את מיקום התאורה:



איור 54 מבנה המערכת עם הטשטוש

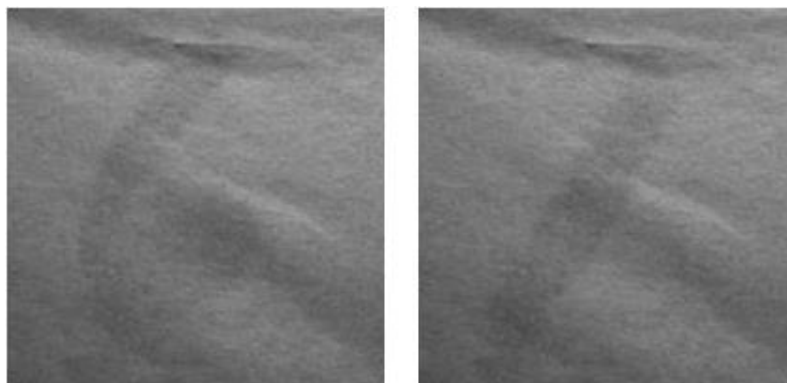
כפי שניתן לראות באיור 54 את הנורה מיקמנו במרחק של 28cm, בגובה של 27cm ובזווית של 45°.

את התמונות הבאות קיבלנו לאחר הדגימה (איור 55) :



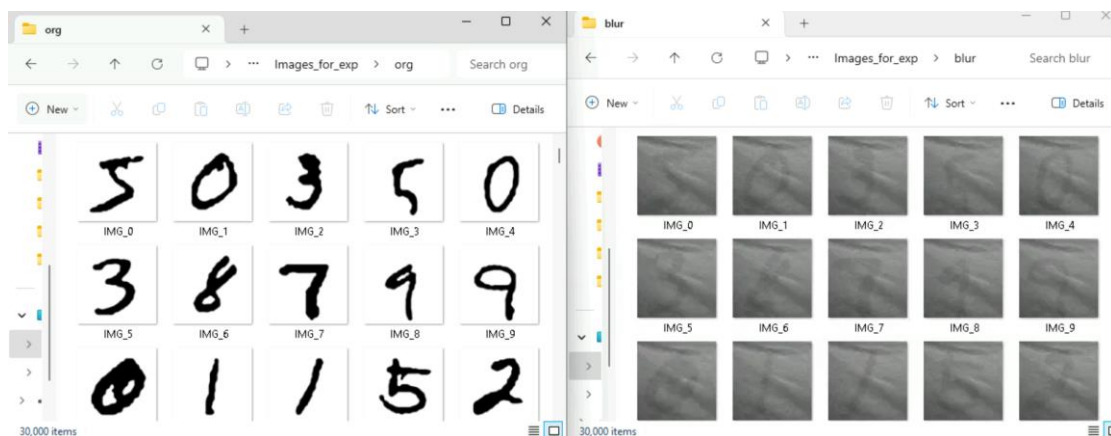
איור 55 התמונות לאחר הדגימה

בשלב הבא, ביצענו חיתוך לתמונות על-מנת להסיר את חלקי המסך שאינם רלוונטיים. לאחר-מכן, המרנו את התמונות ל-`mode=L` ו-`resize` ל-`200x200` פיקסלים. כפי שניתן לראות באיור 56 נקבל:



איור 56 התמונות לאחר ביצוע חיתוך ל-`200x200`

לאחר-מכן הכנסנו את התמונות לרשת ה-`Encoder-Decoder`, כאשר באותו מיקום בשתי תיקיות נפרדות (`org` ו-`blur`), תהיינה אותה תמונה עם טשטוש באמצעות רקמה מפזרת ובלי טשטוש, כפי שניתן לראות באיור 57 :

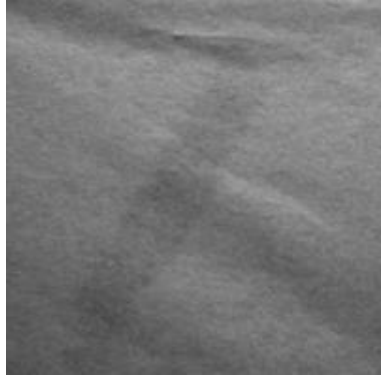


איור 57 הכנסת התמונות לרשת כאשר ה-`org` וה-`blur` באותו מיקום

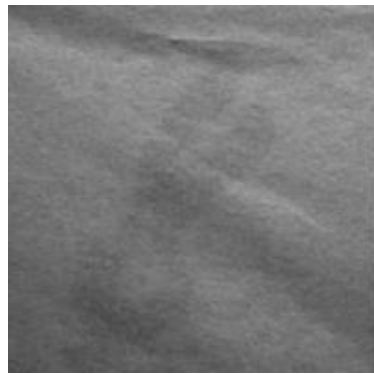
5.6.2 תוצאות ומסקנות

בשלב הבא, העברנו את כלל התמונות דרך הרשת של ה-Encoder-Decoder כדי לוודא שהרשת אכן מצליחה לפענח את התמונות המטושטשות.

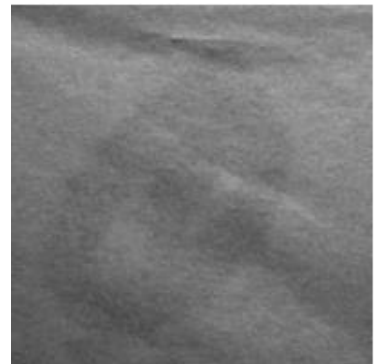
דוגמאות לתמונות שהרשת הצליחה לזהות (איור 58, 59, 60, 61) :



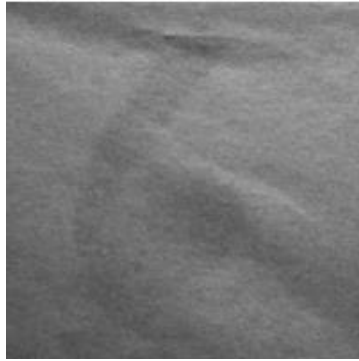
איור 58



איור 59



איור 60



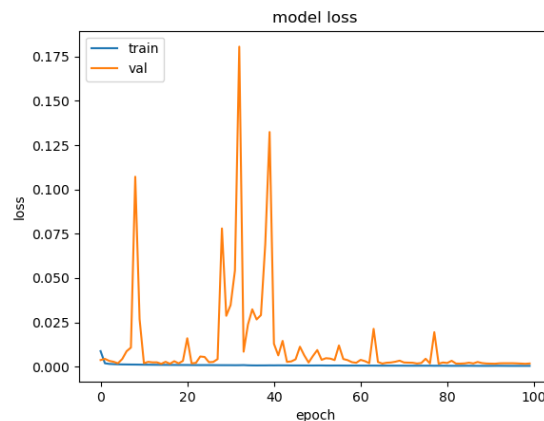
איור 61 באיור זה ובאיורים 58, 59 ו-60 ניתן לראות בשמאל: את התמונה המטושטשת, ובימין: את פענוח התמונה

בתמונות ניתן לראות כי הרשת הצליחה לפענח את התמונות בצורה טובה ולהסיר את הטשטוש.

להלן יוצגו הגרפים של תוצאות הרשת:

פונקציית Loss:

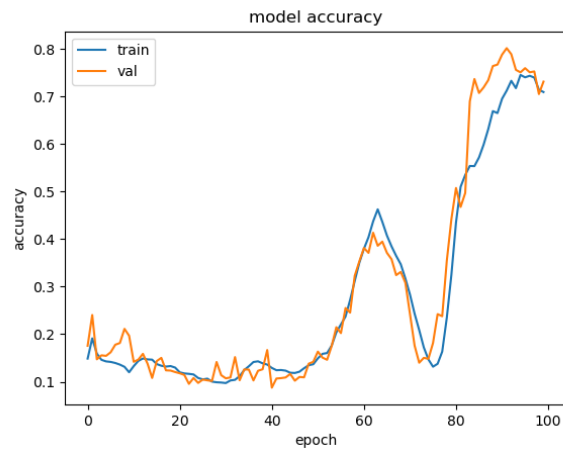
ניתן לראות בגרף 5, שלמרות שישנה אי-יציבות בגרף ה-Loss ישנה התכנסות גם ב-Train וגם ב-Validation ולכן המודל כן עובד כראוי.



גרף 5 model loss in practical

Accuracy:

ניתן לראות בגרף 6, שמבחינת הדיוק ה-Validation עוקב אחר ה-Train מה שמעיד על מודל שעובד בצורה כללית טובה למדי.



גרף 6 model accuracy in practical

פרק 6: סיכום ומבט לעתיד

מטרת פרויקט זה הייתה לפתח מערכת אשר תהיה מסוגלת לפענח תמונות מטושטשות הנגרמות על-ידי מדיום מפזר, כחלופה בריאה יותר לתצלומי רנטגן מסורתיות. הפרויקט ממנף ספקטרוסקופיה קרובה לאינפרא-אדום (NIR), המשתמשת באורכי גל ארוכים יותר מקרני הרנטגן, מה שמבטיח את בטיחות המטופל תוך שמירה על דיוק ההדמיה.

על-ידי פענוח מוצלח של התמונות המטושטשות, המערכת מדגימה את הפוטנציאל לשימוש בגלי NIR בהדמיה רפואית כדי לספק חלופה בטוחה יותר לקרני הרנטגן, עם השלכות רחבות על אבחון לא פולשני.

פרויקט זה מניח את הבסיס לגל חדש של טכנולוגיות הדמיה רפואית לא פולשנית. ככל שהספקטרוסקופיה של NIR מתקדמת, נוכל לראות שיפורים נוספים ברזולוציה ובבהירות התמונה, מה שהופך אותה לתחליף בר-קיימא להדמיית רנטגן באזורים כמו צילומי שיניים, עצמות ורקמות רכות. גרסאות עתידיות של המערכת יכולות למנף GPU's חזקים יותר, רשתות נוירונים עמוקות יותר ומערכי נתונים גדולים יותר, כדי להתמודד עם מבני רקמה מורכבים יותר ודפוסי פיזור.

בטווח הארוך, גישה זו יכולה להתרחב ליישומים ביו-רפואיים אחרים כגון ניטור איברים פנימיים או גילוי גידולים בשלב מוקדם. על-ידי חידוד נוסף של האלגוריתמים ואופטימיזציה של החומרה, הדמיה מבוססת NIR יכולה להפוך לכלי חיוני בטיפול רפואי ברפואה מתקדמת.

ביבליוגרפיה

1. Novelline, Robret (1997). Squire's Fundamentals of Radiology. Harvard University Press. 5th edition. ISBN 0-674-83339-2.
2. "RTAB: the Rayleigh scattering database". Lynn Kissel. 2000-09-02. Archived from the original on 2011-12-12. Retrieved 2012-11-08.
3. "An introduction to near infrared (NIR) spectroscopy | IM Publications Open". www.impopen.com. Retrieved 2022-06-01.
4. Yamaguchi, Kouichi; Sakamoto, Kenji; Akabane, Toshio; Fujimoto, Yoshiji (November 1990). [A Neural Network for Speaker-Independent Isolated Word Recognition](#). First International Conference on Spoken Language Processing (ICSLP 90). Kobe, Japan. Archived from the original on 2021-03-07. Retrieved 2019-09-04.
5. Bin Ding, Huimin Qian, Jun Zhou, [Activation functions and their characteristics in deep neural networks](#), IEEE, 2018-06, pp. 1836-1841 doi: 10.1109/CCDC.2018.8407425.
6. Xavier Glorot; Antonie Bordes; Yoshua Bengio (2011). [Deep sparse rectifier neural networks](#). AISTATS. Rectifier and softplus activation functions. The second one is a smooth version of the first.
7. Andrew L. Maas, Awni Y. Hannun, Andrew Y. Ng (2014). [Rectifier Nonlinearities Improve Neural Network Acoustic Models](#).
8. He, Kaiming; Zhang, Xiangyu; Ren, Shaoqing; Sun, Jian (2015). "Delving Deep into Rectifiers: Surpassing Human-Level Performance on Image Net Classification". [arXiv: 1502.01852 \[cs.CV\]](#).
9. Ling, Yibei; He, Bin (December 1993). "[Entropic analysis of biological growth models](#)". [IEEE Transactions on Biomedical Engineering](#). 40 (12): 1193-2000. doi: 10.1109/10.250574. PMID 8125495.
10. Ioffe, Sergey; Szegedy, Christian (2015). "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". [arXiv: 1502.03167 \[cs.LG\]](#).
11. Hastie, Trevor; Tibshirani, Robret; Friedman, Jerome H. (2001). [The Elements of Statistical Learning](#). Springer. p. 18. ISBN 0-387-95284-5.
12. John pomerat, Aviv Segev, and Rituparna Datta, On Neural Network Activation Functions and Optimizers in Relation to Polynomial Regression, 2019 IEEE International Conference on Big Data (Big Data).

Abstract

This project explores a new approach to medical imaging by developing a system that aims to be an effective replacement for X-rays in NIR waves. X-rays, despite being very effective in creating clear images for diagnosis, pose risks to the patient's health due to their short wavelengths and high energy.

NIR waves, which have longer wavelengths and significantly lower energy, provide a safer alternative, but their limited penetration causes the images to be blurred, an effect that makes them less useful for medical professionals, due to difficulty in deciphering the findings. In order to be able to overcome this challenge, we built a system that will take the blurry images and decode them into original and clear images by the Encoder-Decoder network.

The Encoder-Decoder network at the theoretical level, it is a neural network that takes information from the input image and compresses it into compact information, called a Feature Vector, by taking the dominant values of the pixels. So that the information is not lost in the compression of the information, we added a Skip Connection to the network, which connects each block of the Encoder part to the corresponding block in the Decoder part. The Decoder part takes the feature vector and restores it to the original image.

The dataset in the network was split into three groups: Train, Validation and Test to optimize and verify the performance. In all processes, the network uses standard deep learning techniques such as: the ReLU and Sigmoid activation functions, in the convolution operations, the Kernel matrix, Batch Normalization, the Loss function and the Adam optimizer.

After we wrote a code that implements the Encoder-Decoder network, we built a computer simulation for the system that will simulate the blurry photos, and when we put the photos into the neural network, we will get the blurring removed.

To put this into practice, we took a closed box to simulate laboratory conditions and placed an optical surface inside it to mount the Basler camera on, so that it would be at 90° to the Waveshare screen. We fixed the screen with the help of appropriate legs which we printed on a 3D printer. We also used Led lighting so that the photos would

be clear, and we placed it in a position that would maximally prevent light reflection towards the camera.

In order for the system to work properly, we used a GPU processor which is designed to handle image processing tasks, a Raspberry Pi because we needed a compact computer that we could put in a box and a UART serial communication for there to be communication between the Waveshare screen and the Raspberry Pi. In addition, in order to have enough electric current to power the camera and the Waveshare screen, we used a USB 3.0 splitter that we connected to the Raspberry Pi.

We performed the experiment using the MNIST images intended for deep learning. Before performing the actual experiment, we did a simulation that simulated blurring by writing a suitable code and then we inserted the blurred images into the neural network to see if the network would be able to remove the blurring.

After we saw that the simulation experiment did work, we performed the actual experiment when we did the blurring using a tissue that we placed on the Waveshare screen, which is supposed to simulate scattering tissue in the real world and we started sampling images. After sampling the images, we took care of closing the resources for memory management, security, collision prevention, etc. We entered the blurred images into the neural network to see if the network would be able to decode the images.

In the experiment, we discovered that the network did manage to remove the blur quite well, both in terms of the decoded images and in terms of the graphs modeling the Loss function, when you can see from them that the Validation follows the Train accurately.

From this experiment we concluded that taking photographs for patients using longer wavelengths than X-rays holds promise as a safer alternative to X-rays in medical diagnosis. Potential future applications of this technology may include early detection of tumors and other medical advances as the system and neural network technology continue to improve.

NIR-based imaging can be an essential tool in medical treatment in advanced medicine.

Note: In our project we did not use NIR waves but wavelengths of the Basler camera.

Table of Contents

Abstract (Hebrew).....	x
Part 1: Introduction.....	1
Part 2: Encoder-Decoder network structure.....	3
2.1 The distribution of data	3
2.1.1 MaxPooling2D	3
2.1.2 Activation function.....	5
2.2 The convolution operation	9
2.2.1 Kernel matrix.....	9
2.2.2 Batch Normalization.....	12
2.2.3 Loss function	13
2.2.4 Optimizer Adam	13
Part 3: Practical application of the Encoder-Decoder network.....	15
3.1 The Encoder part	15
3.2 The Decoder part.....	16
Part 4: First experiment- simulation	20
4.1 GPU.....	20
4.1.1 Installing the GPU	21
4.2 Dataset MNIST	22
4.3 Blur simulation.....	23
Part 5: Second experiment- practical application of the system	27
5.1 RPI Module 4B.....	27
5.1.1 Installing an operating system for the RPI.....	28
5.2 UART communication model	30
5.2.1 Installing the UART driver.....	31
5.3 The Waveshare screen.....	31
5.3.1 Installing the EPSCOMM software	33
5.3.2 Preparation of information- switching to BMP format	38
5.3.3 Display images on the screen.....	40
5.3.4 Closing resources.....	45
5.3.5 Fixation to the Waveshare screen	46
5.4 Basler camera	47
5.4.1 Installing the Pylon software	49
5.5 Lighting.....	50
5.6 The course of the experiment	50
5.6.1 Diffuse tissue	50
5.6.2 Results and conclusions.....	53
Part 6: Summary and look to the future	56
Bibliography.....	57
Abstract (English)	i

This work was carried out under the supervision of **Dr Yossef Danan**
Department of Electrical Engineering, Jerusalem College of Engineering,
Jerusalem.



Azrieli College of Engineering Jerusalem
Department of Electrical Engineering

Imaging Through Scattering Tissue

Submitted in Partial Fulfillment
Of the Requirements for the Degree of
Bachelor of Science in Engineering

By
Ilai Gam Zu Letova
Uriya Avdar



Azrieli College of Engineering Jerusalem
Department of Electrical Engineering

Imaging Through Scattering Tissue

Submitted in Partial Fulfillment
Of the Requirements for the Degree of
Bachelor of Science in Engineering

By
Ilai Gam Zu Letova
Uriya Avdar